

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	7
1 ПОСТАНОВКА ЗАДАЧИ.....	8
1.1 Описание входных данных.....	10
1.2 Описание выходных данных.....	12
2 МЕТОД РЕШЕНИЯ.....	14
3 ОПИСАНИЕ АЛГОРИТМОВ.....	19
3.1 Алгоритм метода set_connect класса cl_base.....	19
3.2 Алгоритм метода delete_connect класса cl_base.....	20
3.3 Алгоритм метода emit_signal класса cl_base.....	21
3.4 Алгоритм метода get_absolute_path класса cl_base.....	22
3.5 Алгоритм метода get_class_number класса cl_base.....	23
3.6 Алгоритм деструктора класса cl_base.....	24
3.7 Алгоритм метода get_class_number класса cl_application.....	24
3.8 Алгоритм метода signal_cl_1 класса cl_application.....	25
3.9 Алгоритм метода handler_cl_1 класса cl_application.....	25
3.10 Алгоритм метода build_tree_objects класса cl_application.....	26
3.11 Алгоритм метода exec_app класса cl_application.....	29
3.12 Алгоритм метода get_class_number класса cl_2.....	33
3.13 Алгоритм метода signal_cl_2 класса cl_2.....	33
3.14 Алгоритм метода handler_cl_2 класса cl_2.....	33
3.15 Алгоритм метода get_class_number класса cl_3.....	34
3.16 Алгоритм метода signal_cl_3 класса cl_3.....	34
3.17 Алгоритм метода handler_cl_3 класса cl_3.....	35
3.18 Алгоритм метода get_class_number класса cl_4.....	35
3.19 Алгоритм метода signal_cl_4 класса cl_4.....	36
3.20 Алгоритм метода handler_cl_4 класса cl_4.....	36

3.21 Алгоритм метода get_class_number класса cl_5.....	37
3.22 Алгоритм метода signal_cl_5 класса cl_5.....	37
3.23 Алгоритм метода handler_cl_5 класса cl_5.....	38
3.24 Алгоритм метода get_class_number класса cl_6.....	38
3.25 Алгоритм метода signal_cl_6 класса cl_6.....	39
3.26 Алгоритм метода handler_cl_6 класса cl_6.....	39
3.27 Алгоритм функции class_number_to_signal.....	40
3.28 Алгоритм функции class_number_to_handler.....	41
3.29 Алгоритм функции enable_all.....	42
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	43
5 КОД ПРОГРАММЫ.....	66
5.1 Файл cl_2.cpp.....	66
5.2 Файл cl_2.h.....	67
5.3 Файл cl_3.cpp.....	67
5.4 Файл cl_3.h.....	68
5.5 Файл cl_4.cpp.....	68
5.6 Файл cl_4.h.....	69
5.7 Файл cl_5.cpp.....	70
5.8 Файл cl_5.h.....	70
5.9 Файл cl_6.cpp.....	71
5.10 Файл cl_6.h.....	72
5.11 Файл cl_application.cpp.....	72
5.12 Файл cl_application.h.....	79
5.13 Файл cl_base.cpp.....	80
5.14 Файл cl_base.h.....	90
5.15 Файл main.cpp.....	92
6 ТЕСТИРОВАНИЕ.....	94

ЗАКЛЮЧЕНИЕ.....	98
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	99

ВВЕДЕНИЕ

Настоящая курсовая работа выполнена в соответствии с требованиями ГОСТ Единой системы программной документации (ЕСПД) [1]. Все этапы решения задач курсовой работы фиксированы, соответствуют требованиям, приведенным в методическом пособии для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [2-3] и методике разработки объектно-ориентированных программ [4-6].

Актуальность работы обусловлена тем, что современное программное обеспечение часто строится как система взаимосвязанных и взаимодействующих между собой объектов. Умение разрабатывать такие системы, организовывать связи между объектами и управлять их состоянием является важной частью профессиональной подготовки в области программирования.

Целью работы является получение практических навыков в области объектно-ориентированного программирования с использованием языка C++. Объектно-ориентированное программирование (ООП) - это подход, при котором программа строится из взаимодействующих объектов, каждый из которых обладает своими данными и поведением.

Для достижения цели в ходе выполнения последовательных курсовых работ (от первой до четвёртой версии) решались задачи по постепенному усложнению программной системы: от простого построения иерархического дерева объектов до реализации полноценного механизма обмена сигналами с передачей текстовых сообщений между объектами.

1 ПОСТАНОВКА ЗАДАЧИ

Реализовать механизм взаимодействия объектов с использованием сигналов и обработчиков, с передачей вместе сигналом текстового сообщения (строковой переменной).

Для организации взаимосвязи по механизму сигналов и обработчиков в базовый класс добавить три метода:

- установления связи между сигналом текущего объекта и обработчиком целевого объекта;
- удаления (разрыва) связи между сигналом текущего объекта и обработчиком целевого объекта;
- выдачи сигнала от текущего объекта с передачей строковой переменной. Включенный объект может выдать или обработать сигнал.

Методу установки связи передать адрес метода сигнала текущего объекта, адрес целевого объекта и адрес метода обработчика целевого объекта.

Методу удаления (разрыва) связи передать адрес метода сигнала текущего объекта, адрес целевого объекта и адрес метода обработчика целевого объекта.

Методу выдачи сигнала передать адрес метода сигнала и строковую переменную. В данном методе реализовать алгоритм:

1. Если текущий объект отключен, то выход, иначе к пункту 2.
2. Вызов метода сигнала с передачей строковой переменной по ссылке.
3. Цикл по всем связям сигнал-обработчик текущего объекта:
 - 3.1. Если в очередной связи сигнал-обработчик участвует метод сигнала, переданный по параметру, то проверить готовность целевого объекта. Если целевой объект готов, то вызвать метод обработчика целевого объекта указанной в связи и передать в качестве аргумента строковую переменную по значению.

4. Конец цикла.

Для приведения указателя на метод сигнала и на метод обработчика использовать параметризированное макроопределение препроцессора.

В базовый класс добавить метод определения абсолютной пути до текущего объекта. Этот метод возвращает абсолютный путь текущего объекта.

Состав и иерархия объектов строится посредством ввода исходных данных. Ввод организован как в версии № 3 курсовой работы. Если при построении дерева иерархии возникает ситуация дуближа имен среди починенных у текущего головного объекта, то новый объект не создается.

Система содержит объекты шести классов с номерами: 1, 2, 3, 4, 5, 6. Классу корневого объекта соответствует номер 1. В каждом производном классе реализовать один метод сигнала и один метод обработчика.

Каждый метод сигнала с новой строки выводит:

Signal from «абсолютная координата объекта»

Каждый метод сигнала добавляет переданной по параметру строке текста номер класса принадлежности текущего объекта по форме:

«пробел»(class: «номер класса»)

Каждый метод обработчика с новой строки выводит:

Signal to «абсолютная координата объекта» Text: «переданная строка»

Моделировать работу системы, которая выполняет следующие команды с параметрами:

- EMIT «координата объекта» «текст» – выдает сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – устанавливает связь;
- DELETE_CONNECT «координата объекта выдающего сигнал»

«координата целевого объекта» – удаляет связь;

- SET_CONDITION «координата объекта» «значение состояния» – устанавливает состояние объекта.
- END – завершает функционирование системы (выполнение программы).

Реализовать алгоритм работы системы:

- в методе построения системы:
 - о построение дерева иерархии объектов согласно вводу;
 - о ввод и построение множества связей сигнал-обработчик для заданных пар объектов.
- в методе отработки системы:
 - о привести все объекты в состоянии готовности;
 - о цикл:
 - ввод наименования команды;
 - отработка команды.
 - о конец цикла.

Допускаем, что все входные данные вводятся синтаксически корректно. Контроль корректности входных данных можно реализовать для самоконтроля работы программы. Не оговоренные, но необходимые функции и элементы классов добавляются разработчиком.

1.1 Описание входных данных

В методе построения системы.

Множество объектов, их характеристики и расположение на дереве иерархии. Структура данных для ввода согласно изложенному в версии № 3 курсовой работы.

После ввода состава дерева иерархии построчно вводится:

«координата объекта выдающего сигнал» «координата целевого объекта»

Ввод информации для построения связей завершается строкой, которая содержит:

«end_of_connections»

В методе запуска (отработки) системы построчно вводятся множество команд в производном порядке:

- EMIT «координата объекта» «текст» – выдать сигнал от заданного по координате объекта;
- SET_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – установка связи;
- DELETE_CONNECT «координата объекта выдающего сигнал» «координата целевого объекта» – удаление связи;
- SET_CONDITION «координата объекта» «значение состояния» – установка состояния объекта.
- END – завершить функционирование системы (выполнение программы).

Команда END присутствует обязательно.

Если координата объекта задана некорректно, то соответствующая операция не выполняется и с новой строки выдается сообщение об ошибке.

Если не найден объект по координате:

Object «координата объекта» not found

Если не найден целевой объект по координате:

Handler object «координата целевого объекта» not found

Пример ввода:

```

appls_root
/ object_s1 3
/ object_s2 2
/object_s2 object_s4 4
/ object_s13 5
/object_s2 object_s6 6
/object_s1 object_s7 2
endtree
/object_s2/object_s4 /object_s2/object_s6
/object_s2 /object_s1/object_s7
/ /object_s2/object_s4
/object_s2/object_s4 /
end_of_connections
EMIT /object_s2/object_s4 Send message 1
EMIT /object_s2/object_s4 Send message 2
EMIT /object_s2/object_s4 Send message 3
EMIT /object_s1 Send message 4
END

```

1.2 Описание выходных данных

Первая строка:

Object tree

Со второй строки вывести иерархию построенного дерева.

Далее, построчно, если отработал метод сигнала:

Signal from «абсолютная координата объекта»

Если отработал метод обработчика:

Signal to «абсолютная координата объекта» Text: «переданная строка»

Пример вывода:

```

Object tree
appls_root
  object_s1
    object_s7
  object_s2
    object_s4
    object_s6
  object_s13
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text:  Send message 1 (class: 4)
Signal to / Text:  Send message 1 (class: 4)
Signal from /object_s2/object_s4
Signal to /object_s2/object_s6 Text:  Send message 2 (class: 4)
Signal to / Text:  Send message 2 (class: 4)
Signal from /object_s2/object_s4

```

Signal to /object_s2/object_s6 Text: Send message 3 (class: 4)
Signal to / Text: Send message 3 (class: 4)
Signal from /object_s1

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- функция `class_number_to_signal` для получения указателя на метод сигнала по номеру класса;
- функция `class_number_to_handler` для получения указателя на метод обработчика по номеру класса;
- функция `enable_all` для установки состояния готовности для всех объектов на 1;
- параметризованные макроопределения препроцессоров `SIGNAL_D` и `HENDLER_D` для приведения указателя на метод сигнала и на метод обработчика соответственно;
- структура `o_sh`, представляющая собой пользовательский тип данных для хранения соединений "сигнал-обработчик".

Класс `cl_base`:

- свойства/поля:
 - поле динамический массив для хранения адресов экземпляров структуры `o_sh`, содержащих указатель на метод сигнала, указатель на целевой объект и указатель на метод обработчика:
 - наименование — `connects`;
 - тип — вектор указателей на структуру `o_sh`;
 - модификатор доступа — `private`;
- функционал:
 - метод `set_connect` — метод установления связи между сигналом текущего объекта и обработчиком целевого объекта;
 - метод `delete_connect` — метод удаления (разрыва) связи между сигналом текущего объекта и обработчиком целевого объекта;

- о метод `emit_signal` — метод выдачи сигнала от текущего объекта с передачей строковой переменной, включенный объект может выдать или обработать сигнал;
- о метод `get_absolute_path` — метод определения абсолютного пути до текущего объекта;
- о метод `get_class_number` — виртуальный метод для получения номера класса объекта;
- о метод `~cl_base` — деструктор.

Класс `cl_application`:

- функционал:
 - о метод `get_class_number` — виртуальный метод для получения номера класса объекта 1;
 - о метод `signal_cl_1` — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
 - о метод `handler_cl_1` — метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала;
 - о метод `build_tree_objects` — построение исходного дерева иерархии объектов на основе входных данных;
 - о метод `exes_app` — запуск системы, обработка команд над объектами.

Класс `cl_2`:

- функционал:
 - о метод `get_class_number` — виртуальный метод для получения номера класса объекта 2;
 - о метод `signal_cl_2` — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
 - о метод `handler_cl_2` — метод для вывода сообщения "Signal to

<путь> Text: <сообщение>" при получении сигнала.

Класс cl_3:

- функционал:
 - метод get_class_number — виртуальный метод для получения номера класса объекта 3;
 - метод signal_cl_3 — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
 - метод handler_cl_3 — метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Класс cl_4:

- функционал:
 - метод get_class_number — виртуальный метод для получения номера класса объекта 4;
 - метод signal_cl_4 — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
 - метод handler_cl_4 — метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Класс cl_5:

- функционал:
 - метод get_class_number — виртуальный метод для получения номера класса объекта 5;
 - метод signal_cl_5 — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
 - метод handler_cl_5 — метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Класс cl_6:

- функционал:

- о метод `get_class_number` — виртуальный метод для получения номера класса объекта 6;
- о метод `signal_cl_6` — метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса;
- о метод `handler_cl_6` — метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base			Базовый класс для всех объектов системы, обеспечивающий возможность построения иерархического дерева объектов	
		cl_application	public		2
		cl_2	public		3
		cl_3	public		4
		cl_4	public		5
		cl_5	public		6
		cl_6	public		7
2	cl_application			Класс, производный от базового cl_base, являющийся корневым в дереве иерархии	
3	cl_2			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии второго класса	
4	cl_3			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии третьего	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
				класса	
5	cl_4			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии четвёртого класса	
6	cl_5			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии пятого класса	
7	cl_6			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии шестого класса	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `set_connect` класса `cl_base`

Функционал: метод установления связи между сигналом текущего объекта и обработчиком целевого объекта.

Параметры: `TYPE_SIGNAL`, `p_signal`, указатель на метод сигнала текущего объекта; указатель на объект базового класса, `p_object`, содержит адрес целевого объекта; `TYPE_HANDLER`, `p_ob_hendler`, указатель на метод обработчика целевого объекта.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `set_connect` класса `cl_base`

№	Предикат	Действия	№ перехода
1		инициализировать целочисленную переменную <code>i</code> нулевым значением	2
2	значение индекса <code>i</code> меньше количества связей текущего объекта		3
			4
3	в массиве связей текущего объекта уже есть связь со значениями полей, равными значениям переданных		∅

№	Предикат	Действия	№ перехода
	параметров		
		инкрементировать i	2
4		инициализировать указатель на объект структуры связи p_value адресом динамически созданного объекта структуры o_sh	5
5		присвоить полю p_signal объекта по указателю p_value значение входного параметра p_signal	6
6		присвоить полю p_cl_base объекта по указателю p_value значение входного параметра p_object	7
7		присвоить полю p_hendler объекта по указателю p_value значение входного параметра p_ob_hendler	8
8		добавить значение указателя p_value в массив адресов связей текущего объекта	∅

3.2 Алгоритм метода delete_connect класса cl_base

Функционал: метод удаления (разрыва) связи между сигналом текущего объекта и обработчиком целевого объекта.

Параметры: TYPE_SIGNAL, p_signal, указатель на метод сигнала текущего объекта; указатель на объект базового класса, p_object, содержит адрес целевого объекта; TYPE_HANDLER, p_ob_hendler, указатель на метод обработчика целевого объекта.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода delete_connect класса cl_base

№	Предикат	Действия	№ перехода
1		инициализировать целочисленную переменную i	2

№	Предикат	Действия	№ перехода
		нулевым значением	
2	значение индекса <i>i</i> меньше количества связей текущего объекта		3
			∅
3	в массиве связей текущего объекта уже есть связь со значениями полей, равными значениям переданных параметров	удалить динамически созданную структуру связи, адрес которой хранится в <i>i</i> -том элементе массива связей текущего объекта	4
		инкрементировать <i>i</i>	2
4		удалить из массива связей текущего объекта элемент, хранящий адрес найденной структуры связи	∅

3.3 Алгоритм метода `emit_signal` класса `cl_base`

Функционал: метод выдачи сигнала от текущего объекта с передачей строковой переменной, включенный объект может выдать или обработать сигнал.

Параметры: `TYPE_SIGNAL`, `p_signal`, указатель на метод сигнала текущего объекта; ссылка на строковую переменную `s_command` - передаваемый текст.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода `emit_signal` класса `cl_base`

№	Предикат	Действия	№ перехода
1	текущий объект не готов		∅
			2
2		объявление указателя на метод обработчика	3

№	Предикат	Действия	№ перехода
		p_handler	
3		объявление указателя на объект базового класса p_object для хранения адреса целевого объекта	4
4		вызов метода сигнала по указателю p_signal у текущего объекта с ссылкой на s_command в качестве аргумента	5
5		инициализировать целочисленную переменную i нулевым значением	6
6	значение индекса i меньше количества связей текущего объекта		7
			∅
7	связь по адресу в массиве с индексом i относится к вызываемому сигналу	присвоить указателю p_object адрес целевого объекта p_cl_base по указателю на i-ый элемент массива связей connects	8
			10
8	целевой объект p_object готов	присвоить указателю p_handler адрес целевого объекта p_handler по указателю на i-ый элемент массива связей connects	9
			10
9		вызов метода обработчика по указателю p_handler у целевого объекта по указателю p_object с передаваемым текстом s_command в качестве аргумента	10
10		инкрементировать i	6

3.4 Алгоритм метода get_absolute_path класса cl_base

Функционал: метод определения абсолютного пути до текущего объекта.

Параметры: нет.

Возвращаемое значение: строковый, абсолютный путь до текущего объекта.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода *get_absolute_path* класса *cl_base*

№	Предикат	Действия	№ перехода
1		инициализировать строковую переменную <i>path</i> пустой строкой	2
2		инициализировать указатель на объект базового класса <i>p</i> адресом текущего объекта	3
3	текущий объект корневой	возврат строки "/"	∅
			4
4	текущий объект не корневой	записать в начало переменной <i>path</i> строку "/", имя объекта по указателю <i>p</i> , значение переменной <i>path</i>	5
		возврат значения переменной <i>path</i>	∅
5		присвоить указателю <i>p</i> результат вызова метода <i>get_head</i> по указателю <i>p</i>	4

3.5 Алгоритм метода *get_class_number* класса *cl_base*

Функционал: виртуальный метод для получения номера класса объекта.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *get_class_number* класса *cl_base*

№	Предикат	Действия	№ перехода
1		возврат 0	∅

3.6 Алгоритм деструктора класса `cl_base`

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 7.

Таблица 7 – Алгоритм деструктора класса `cl_base`

№	Предикат	Действия	№ перехода
1		инициализировать целочисленную переменную i нулевым значением	2
2	значение индекса i меньше количества подчинённых объектов текущего объекта	удалить динамически созданный объект, адрес которого хранится в i -том элементе массива подчинённых объектов текущего объекта	3
			4
3		инкрементировать i	2
4		инициализировать целочисленную переменную j нулевым значением	5
5	значение индекса j меньше количества связей текущего объекта	удалить динамически созданную структуру связи, адрес которой хранится в j -том элементе массива связей текущего объекта	6
			Ø
6		инкрементировать j	5

3.7 Алгоритм метода `get_class_number` класса `cl_application`

Функционал: виртуальный метод для получения номера класса объекта 1.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *get_class_number* класса *cl_application*

№	Предикат	Действия	№ перехода
1		возврат 1	Ø

3.8 Алгоритм метода *signal_cl_1* класса *cl_application*

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную *s_text* - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *signal_cl_1* класса *cl_application*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего объекта посредством вызова у него метода <i>get_absolute_path</i>	2
2		добавить в конец строки <i>s_text</i> , переданной по ссылке, подстроку "(class: 1)"	Ø

3.9 Алгоритм метода *handler_cl_1* класса *cl_application*

Функционал: метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Параметры: строковый, *s_text*, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *handler_cl_1* класса *cl_application*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code> , строки "Text: ", значения параметра <code>s_text</code> и символа переноса строки	Ø

3.10 Алгоритм метода *build_tree_objects* класса *cl_application*

Функционал: построение исходного дерева иерархии объектов на основе входных данных.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 11.

Таблица 11 – Алгоритм метода *build_tree_objects* класса *cl_application*

№	Предикат	Действия	№ перехода
1		объявление строковых переменных <code>s_coord</code> и <code>s_sub</code>	2
2		объявление целочисленной переменной <code>class_number</code>	3
3		считать значение из стандартного потока ввода в переменную <code>s_coord</code>	4
4		вызов метода редактирования имени <code>set_name</code> текущего объекта с аргументом <code>s_coord</code>	5
5		считать значение из стандартного потока ввода в переменную <code>s_coord</code>	6
6	значение переменной <code>s_coord</code> равно "endtree"		15
		считать значение из стандартного потока ввода в переменные <code>s_sub</code> и <code>class_number</code>	7

№	Предикат	Действия	№ перехода
7		инициализировать указатель на объект базового класса p_head адресом предполагаемого головного объекта посредством вызова метода get_subobject_by_coordinate с аргументом s_coord	8
8	предполагаемый головной объект не был найден?	вывод на консоль строки "Object tree" и символа переноса строки	9
			12
9		вызов метода print_tree текущего объекта	10
10		вывод на консоль строки "The head object ", значения переменной s_coord, строки " is not found" и символа переноса строки	11
11		установить состояние готовности корневого объекта на 0 как признак ошибки посредством вызова метода set_ready с аргументом 0	∅
12	результат вызова метода get_subobject_by_name по указателю p_head с аргументом s_sub не равен значению нулевого указателя	вывод на консоль строки " Dubbing the names of subordinate objects" и символа переноса строки	5
			13
13	номер класса меньше 2 или больше 6		5
			14
14	значение переменной class_number равно 2	динамически создать объект класса cl_2 с аргументами p_head, s_sub	5
	значение переменной class_number равно 3	динамически создать объект класса cl_3 с аргументами p_head, s_sub	5
	значение переменной class_number равно 4	динамически создать объект класса cl_4 с аргументами p_head, s_sub	5
	значение переменной class_number равно 5	динамически создать объект класса cl_5 с аргументами p_head, s_sub	5

№	Предикат	Действия	№ перехода
	class_number равно 5	аргументами p_head, s_sub	
	значение переменной class_number равно 6	динамически создать объект класса cl_6 с аргументами p_head, s_sub	5
			5
15		объявление строковых переменных s_signal_coord, s_handler_coord	16
16		считать значение из стандартного потока ввода в переменную s_signal_coord	17
17	значение переменной s_signal_coord равно "end_of_connections"		23
		считать значение из стандартного потока ввода в переменную s_handler_coord	18
18		инициализировать указатель на объект базового класса p_signal_object результатом вызова метода get_subobject_by_coordinate с аргументом s_signal_coord текущего элемента	19
19		инициализировать указатель на объект базового класса p_handler_object результатом вызова метода get_subobject_by_coordinate с аргументом s_handler_coord текущего элемента	20
20	целевой объект p_handler_object или объект p_signal_object, выдающий сигнал, не найден		16
		инициализировать указатель на метод сигнала p_sig результатом вызова функции class_number_to_signal с номером класса объекта-источника в качестве аргумента	21

№	Предикат	Действия	№ перехода
21		инициализировать указатель на метод обработчика p_hnd результатом вызова функции class_number_to_handler с номером класса объекта-источника в качестве аргумента	22
22		вызов метода установки связи между объектом-источником и объектом-целью по указателю p_signal_object с указателями p_sig, p_handler_object, p_hnd в качестве аргументов	16
23		вывод на консоль строки "Object tree" и символа переноса строки	24
24		вызов метода print_tree текущего объекта	∅

3.11 Алгоритм метода exes_app класса cl_application

Функционал: запуск системы, обработка команд над объектами.

Параметры: нет.

Возвращаемое значение: целочисленный - код возврата программы, определяющий корректность выполнения работы.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода exes_app класса cl_application

№	Предикат	Действия	№ перехода
1	состояние готовности текущего (корневого) объекта равно 0	возврат 1	∅
			2
2		установка состояния готовности для всех объектов на 1	3
3		объявить строковую переменную command	4

№	Предикат	Действия	№ перехода
4		считать значение из стандартного потока ввода в переменную command	5
5	значение переменной command равно "END"		27
	значение переменной command равно "EMIT"	считать значение из стандартного потока ввода в переменную coordinate	6
	значение переменной command равно "SET_CONNECT" или "DELETE_CONNECT"		13
	значение переменной command равно "SET_CONDITION"		22
			27
6		объявить строковую переменную text	7
7		объявить строковую переменную coordinate	8
8		пропуск одного символа (пробелы после координаты) из стандартного потока ввода посредством вызова метода cin.ignore()	9
9		считать оставшуюся часть строки из стандартного потока ввода в переменную text	10
10		инициализировать указатель на объект базового класса p_emit_object результатом вызова метода get_subobject_by_coordinate с аргументом coordinate текущего объекта	11
11	объект p_emit_object не был найден	вывод на консоль строки "Object ", значения переменной coordinate, строки " not found" и символа переноса строки	4
		инициализировать указатель на метод сигнала	12

№	Предикат	Действия	№ перехода
		p_sig класса объекта-источника	
12		вызов метода emit_signal по указателю p_emit_object с указателем p_sig и строкой text в качестве аргументов	4
13		инициализировать булеву переменную is_delete значением false	14
14	значение переменной command равно "DELETE_CONNECT"	присвоить переменной is_delete значение true	15
			15
15		объявить строковые переменные s_signal_coord и s_handler_coord	16
16		считать значения из стандартного потока ввода в переменные s_signal_coord и s_handler_coord	17
17		инициализировать указатель на объект базового класса p_signal_object результатом вызова метода get_subobject_by_coordinate с аргументом p_signal_coord текущего объекта	18
18	объект p_signal_object, выдающий сигнал, не был найден	вывод на консоль строки "Object ", значения переменной s_signal_coord, строки " not found" и символа переноса строки	4
		инициализировать указатель на объект базового класса p_handler_object результатом вызова метода get_subobject_by_coordinate с аргументом p_handler_coord текущего объекта	19
19	целевой объект p_handler_object не был найден	вывод на консоль строки "Handler object ", значения переменной s_handler_coord, строки " not found" и символа переноса строки	4
		инициализировать указатель на метод сигнала p_sig результатом вызова функции	20

№	Предикат	Действия	№ перехода
		class_number_to_signal с номером класса объекта-источника в качестве аргумента	
20		инициализировать указатель на метод обработчика p_hnd результатом вызова функции class_number_to_handler с номером класса объекта-источника в качестве аргумента	21
21	значение переменной is_delete равно истине	вызов метода удаления связи между объектом-источником и объектом-целью по указателю p_signal_object с указателями p_sig, p_handler_object, p_hnd в качестве аргументов	4
		вызов метода установки связи между объектом-источником и объектом-целью по указателю p_signal_object с указателями p_sig, p_handler_object, p_hnd в качестве аргументов	4
22		объявить строковую переменную coord	23
23		объявить целочисленную переменную state	24
24		считать значения из стандартного потока ввода в переменные coord и state	25
25		инициализировать указатель на объект базового класса p_object результатом вызова метода get_subobject_by_coordinate с аргументом coord текущего объекта	26
26	объект p_object не найден	вывод на консоль строки "Object ", значения переменной coord, строки " not found" и символа переноса строки	4
		вызов функции set_ready по указателю p_object с аргументом state	4
27		возврат 0	∅

3.12 Алгоритм метода `get_class_number` класса `cl_2`

Функционал: виртуальный метод для получения номера класса объекта 2.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода `get_class_number` класса `cl_2`

№	Предикат	Действия	№ перехода
1		возврат 2	Ø

3.13 Алгоритм метода `signal_cl_2` класса `cl_2`

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную `s_text` - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода `signal_cl_2` класса `cl_2`

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code>	2
2		добавить в конец строки <code>s_text</code> , переданной по ссылке, подстроку "(class: 2)"	Ø

3.14 Алгоритм метода `handler_cl_2` класса `cl_2`

Функционал: метод для вывода сообщения "Signal to <путь> Text:

<сообщение>" при получении сигнала.

Параметры: строковый, s_text, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 15.

Таблица 15 – Алгоритм метода handler_cl_2 класса cl_2

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода get_absolute_path, строки "Text: ", значения параметра s_text и символа переноса строки	Ø

3.15 Алгоритм метода get_class_number класса cl_3

Функционал: виртуальный метод для получения номера класса объекта 3.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 16.

Таблица 16 – Алгоритм метода get_class_number класса cl_3

№	Предикат	Действия	№ перехода
1		возврат 3	Ø

3.16 Алгоритм метода signal_cl_3 класса cl_3

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную s_text - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 17.

Таблица 17 – Алгоритм метода *signal_cl_3* класса *cl_3*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code>	2
2		добавить в конец строки <code>s_text</code> , переданной по ссылке, подстроку " <code>(class: 3)</code> "	∅

3.17 Алгоритм метода *handler_cl_3* класса *cl_3*

Функционал: метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Параметры: строковый, `s_text`, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 18.

Таблица 18 – Алгоритм метода *handler_cl_3* класса *cl_3*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code> , строки "Text: ", значения параметра <code>s_text</code> и символа переноса строки	∅

3.18 Алгоритм метода *get_class_number* класса *cl_4*

Функционал: виртуальный метод для получения номера класса объекта 4.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 19.

Таблица 19 – Алгоритм метода *get_class_number* класса *cl_4*

№	Предикат	Действия	№ перехода
1		возврат 4	Ø

3.19 Алгоритм метода *signal_cl_4* класса *cl_4*

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную *s_text* - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 20.

Таблица 20 – Алгоритм метода *signal_cl_4* класса *cl_4*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего объекта посредством вызова у него метода <i>get_absolute_path</i>	2
2		добавить в конец строки <i>s_text</i> , переданной по ссылке, подстроку "(class: 4)"	Ø

3.20 Алгоритм метода *handler_cl_4* класса *cl_4*

Функционал: метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Параметры: строковый, *s_text*, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 21.

Таблица 21 – Алгоритм метода *handler_cl_4* класса *cl_4*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code> , строки "Text: ", значения параметра <code>s_text</code> и символа переноса строки	Ø

3.21 Алгоритм метода *get_class_number* класса *cl_5*

Функционал: виртуальный метод для получения номера класса объекта 5.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 22.

Таблица 22 – Алгоритм метода *get_class_number* класса *cl_5*

№	Предикат	Действия	№ перехода
1		возврат 5	Ø

3.22 Алгоритм метода *signal_cl_5* класса *cl_5*

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную `s_text` - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 23.

Таблица 23 – Алгоритм метода *signal_cl_5* класса *cl_5*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего	2

№	Предикат	Действия	№ перехода
		объекта посредством вызова у него метода <code>get_absolute_path</code>	
2		добавить в конец строки <code>s_text</code> , переданной по ссылке, подстроку " <code>(class: 5)</code> "	Ø

3.23 Алгоритм метода `handler_cl_5` класса `cl_5`

Функционал: метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Параметры: строковый, `s_text`, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 24.

Таблица 24 – Алгоритм метода `handler_cl_5` класса `cl_5`

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code> , строки "Text: ", значения параметра <code>s_text</code> и символа переноса строки	Ø

3.24 Алгоритм метода `get_class_number` класса `cl_6`

Функционал: виртуальный метод для получения номера класса объекта 6.

Параметры: нет.

Возвращаемое значение: целочисленный, номер класса объекта.

Алгоритм метода представлен в таблице 25.

Таблица 25 – Алгоритм метода `get_class_number` класса `cl_6`

№	Предикат	Действия	№ перехода
1		возврат 6	Ø

3.25 Алгоритм метода `signal_cl_6` класса `cl_6`

Функционал: метод для вывода сообщения "Signal from <путь>" и добавления к переданной строке номера класса.

Параметры: ссылка на строковую переменную `s_text` - текст сообщения для метода обработчика.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 26.

Таблица 26 – Алгоритм метода `signal_cl_6` класса `cl_6`

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal from " и абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code>	2
2		добавить в конец строки <code>s_text</code> , переданной по ссылке, подстроку "(class: 6)"	∅

3.26 Алгоритм метода `handler_cl_6` класса `cl_6`

Функционал: метод для вывода сообщения "Signal to <путь> Text: <сообщение>" при получении сигнала.

Параметры: строковый, `s_text`, текст сообщения.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 27.

Таблица 27 – Алгоритм метода `handler_cl_6` класса `cl_6`

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Signal to ", абсолютного пути текущего объекта посредством вызова у него метода <code>get_absolute_path</code> , строки "Text: ", значения параметра <code>s_text</code> и символа переноса строки	∅

3.27 Алгоритм функции class_number_to_signal

Функционал: получение значения указателя на метод сигнала по номеру класса.

Параметры: целочисленный, class_number, номер класса.

Возвращаемое значение: значение указателя типа TYPE_SIGNAL на метод сигнала соответствующего класса или нулевое значение указателя.

Алгоритм функции представлен в таблице 28.

Таблица 28 – Алгоритм функции class_number_to_signal

№	Предикат	Действия	№ перехода
1	значение входного параметра class_number равно 1	возврат адреса метода сигнала класса cl_application	Ø
	значение входного параметра class_number равно 2	возврат адреса метода сигнала класса cl_2	Ø
	значение входного параметра class_number равно 3	возврат адреса метода сигнала класса cl_3	Ø
	значение входного параметра class_number равно 4	возврат адреса метода сигнала класса cl_4	Ø
	значение входного параметра class_number равно 5	возврат адреса метода сигнала класса cl_5	Ø
	значение входного параметра class_number равно 6	возврат адреса метода сигнала класса cl_6	Ø
		возврат нулевого адреса	Ø

3.28 Алгоритм функции class_number_to_handler

Функционал: получение значения указателя на метод обработчика по номеру класса.

Параметры: целочисленный, class_number, номер класса.

Возвращаемое значение: значение указателя типа TYPE_HANDLER на метод обработчика соответствующего класса или нулевое значение указателя.

Алгоритм функции представлен в таблице 29.

Таблица 29 – Алгоритм функции class_number_to_handler

№	Предикат	Действия	№ перехода
1	значение входного параметра class_number равно 1	возврат адреса метода обработчика класса cl_application	Ø
	значение входного параметра class_number равно 2	возврат адреса метода обработчика класса cl_2	Ø
	значение входного параметра class_number равно 3	возврат адреса метода обработчика класса cl_3	Ø
	значение входного параметра class_number равно 4	возврат адреса метода обработчика класса cl_4	Ø
	значение входного параметра class_number равно 5	возврат адреса метода обработчика класса cl_5	Ø
	значение входного параметра class_number равно 6	возврат адреса метода обработчика класса cl_6	Ø
		возврат нулевого адреса	Ø

3.29 Алгоритм функции `enable_all`

Функционал: установка состояния готовности для всех объектов на 1.

Параметры: указатель на объект базового класса, `p_object`, содержит адрес переданного объекта.

Возвращаемое значение: отсутствует.

Алгоритм функции представлен в таблице 30.

Таблица 30 – Алгоритм функции `enable_all`

№	Предикат	Действия	№ перехода
1		установка состояния готовности текущего объекта на 1	2
2		инициализировать целочисленную переменную <code>i</code> единичным значением	3
3	значение индекса <code>i</code> меньше количества подчинённых объектов по указателю <code>p_object</code>	вызов метода <code>enable_all</code> рекурсивно для включения всех подчинённых объектов посредством вызова метода <code>get_subobject_by_index</code> с аргументом <code>i</code> по указателю <code>p_object</code>	4
			Ø
4		инкрементировать <code>i</code>	3

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-23.

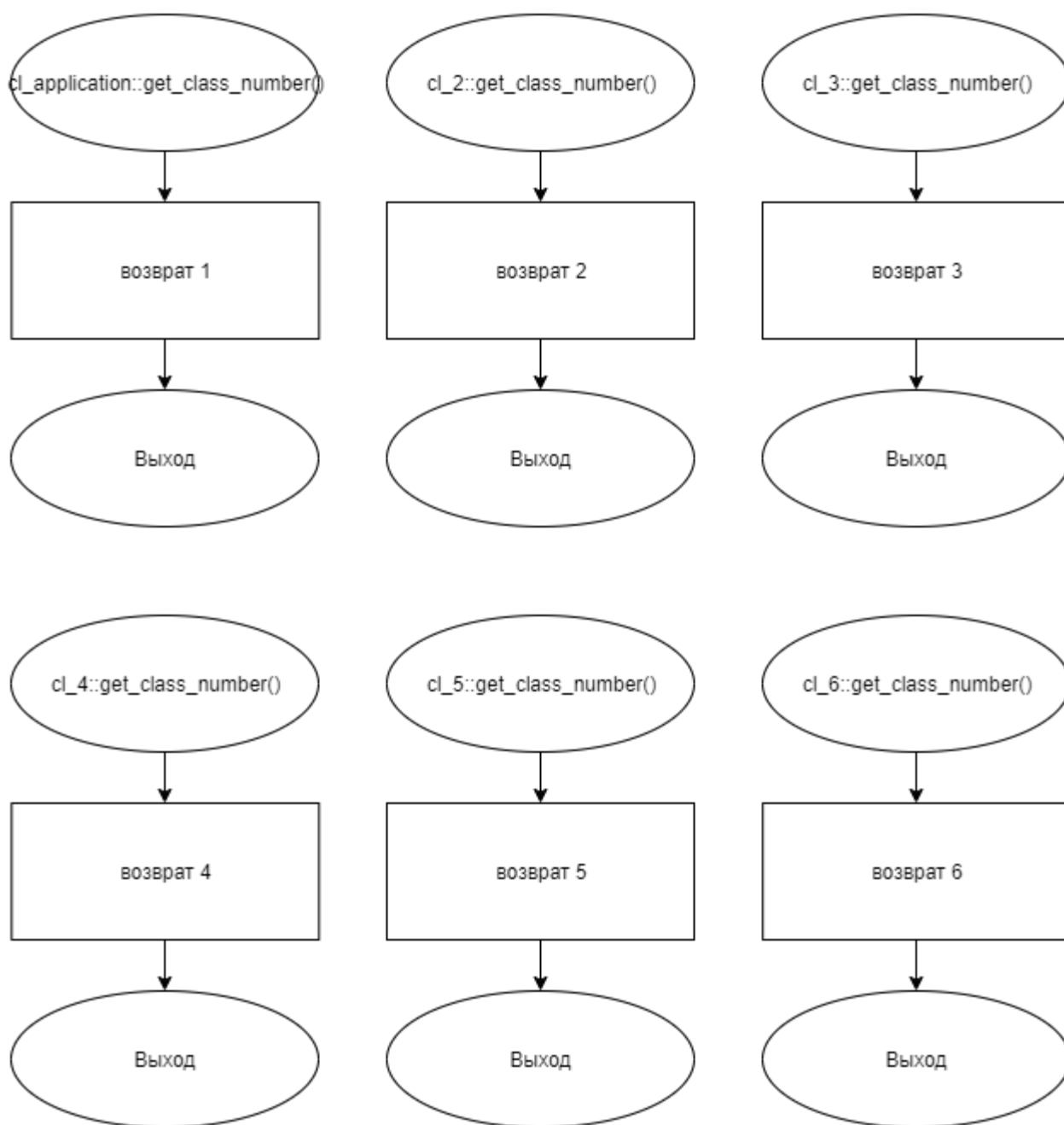


Рисунок 1 – Блок-схема алгоритма

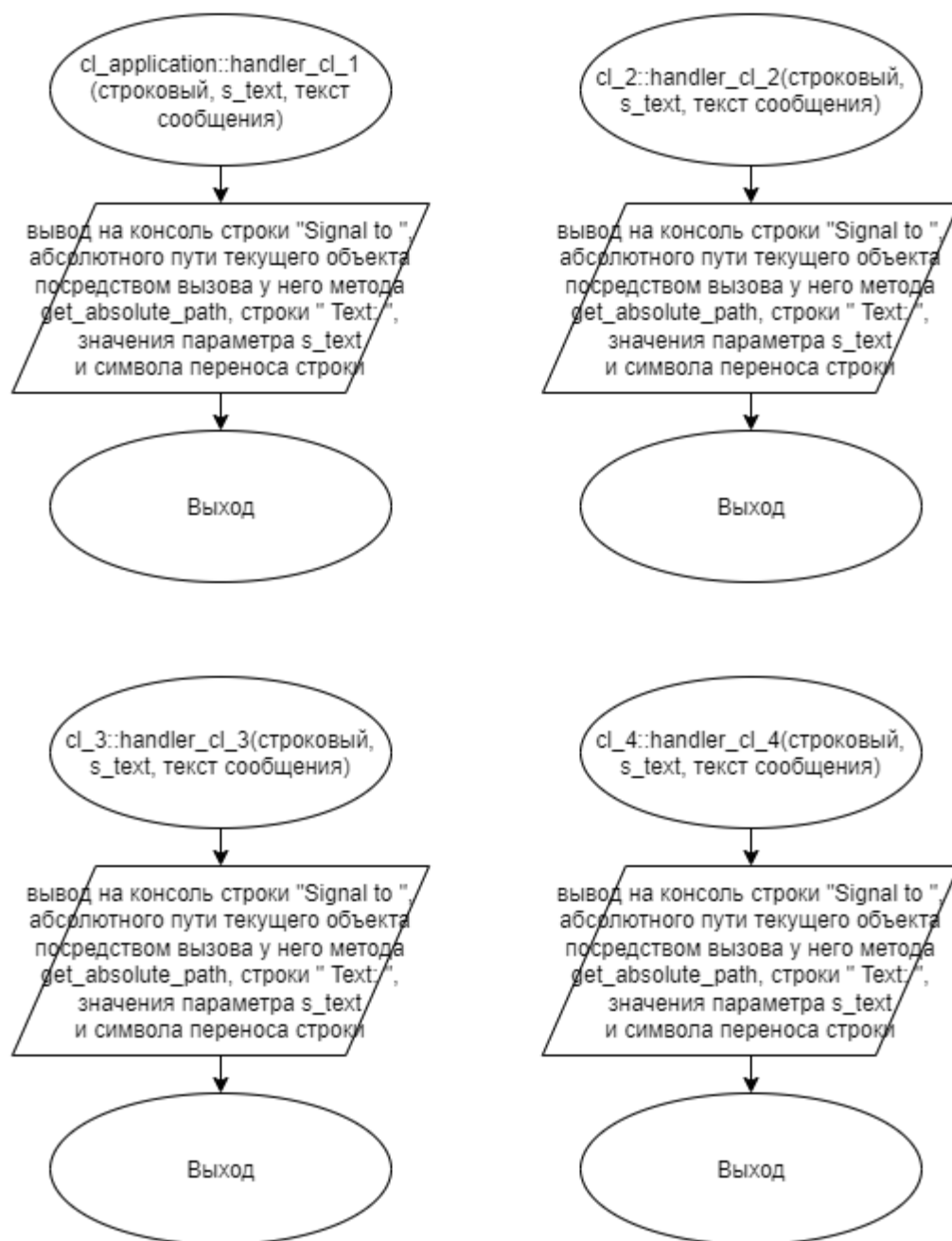


Рисунок 2 – Блок-схема алгоритма

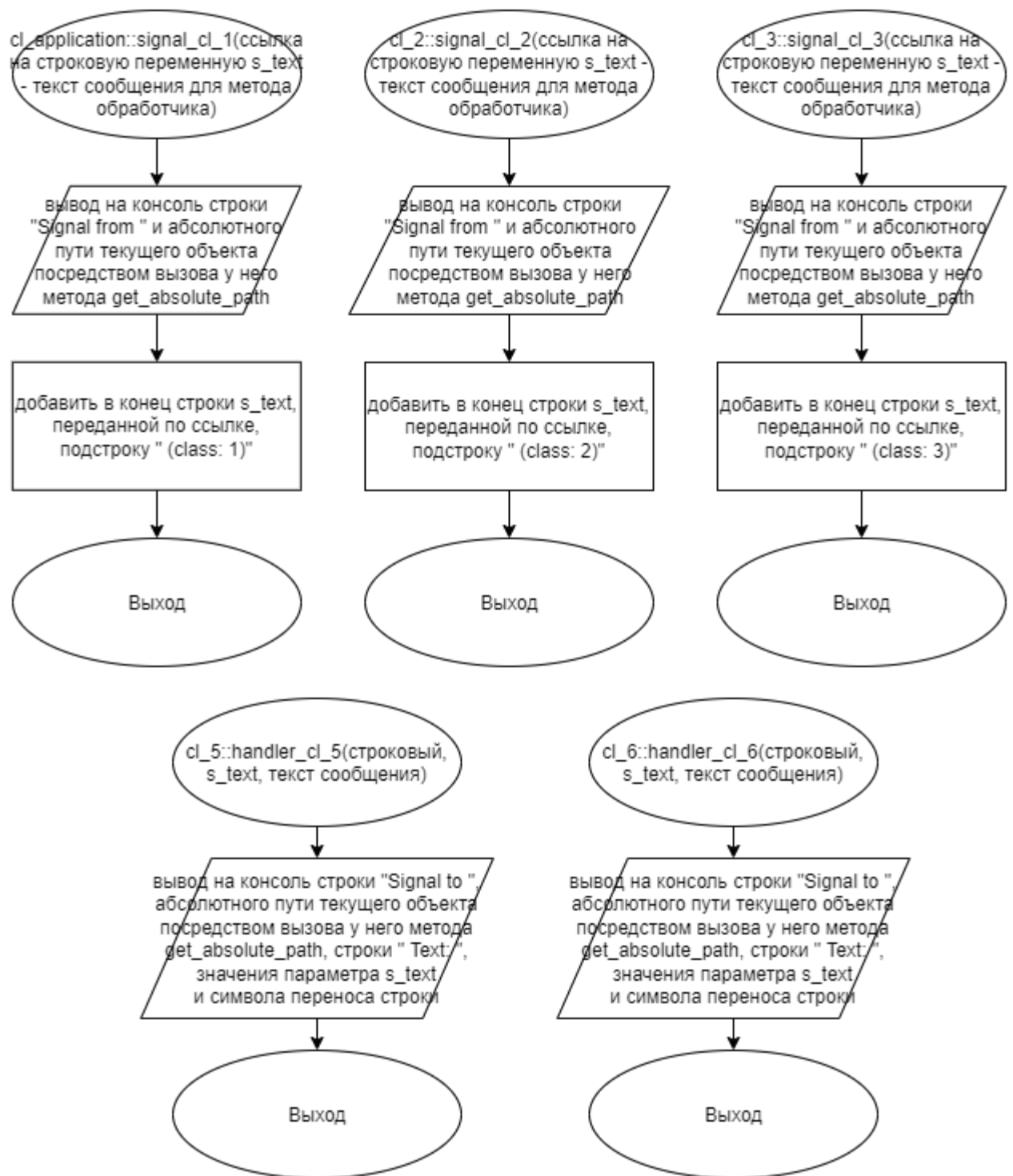


Рисунок 3 – Блок-схема алгоритма

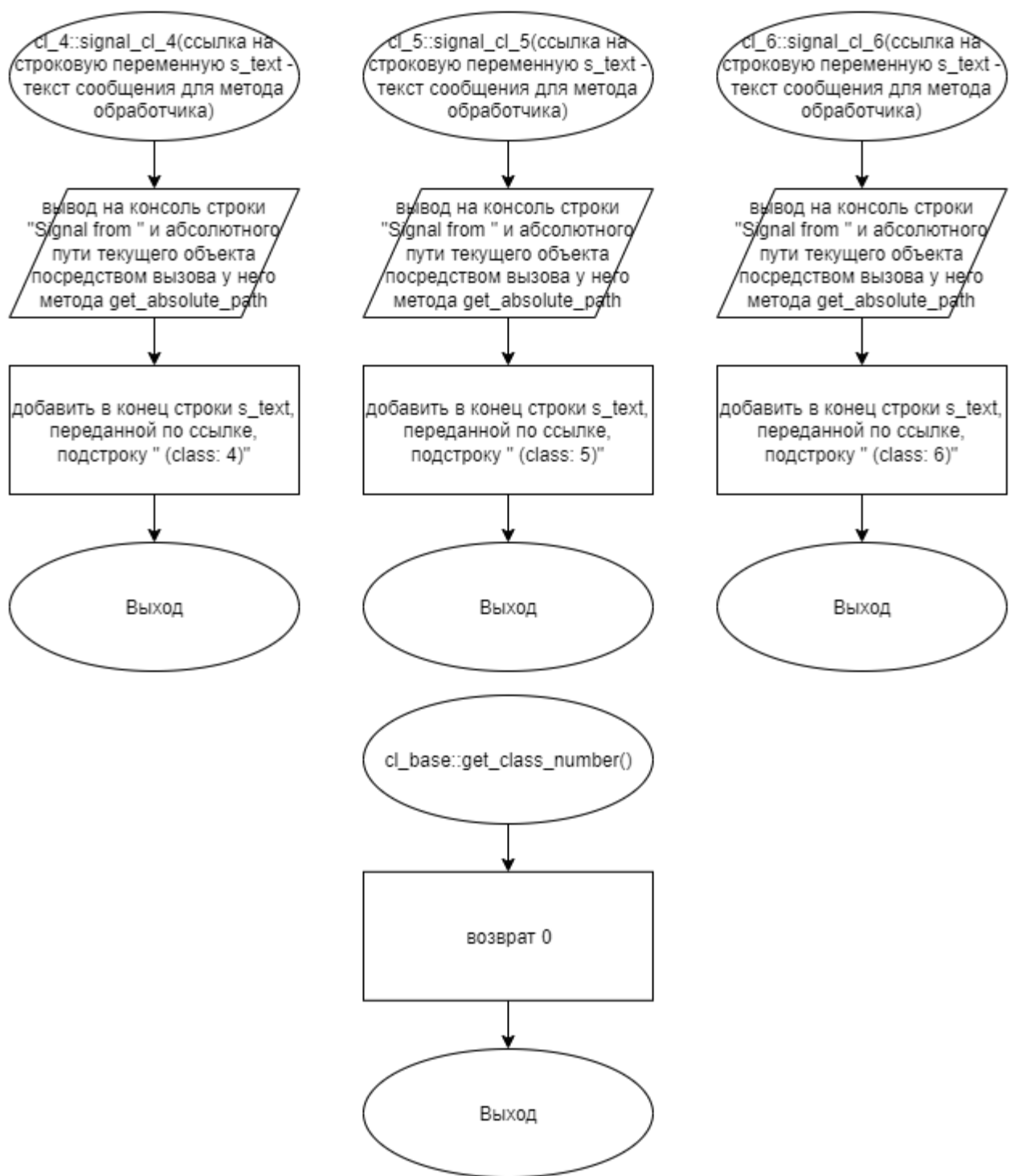


Рисунок 4 – Блок-схема алгоритма

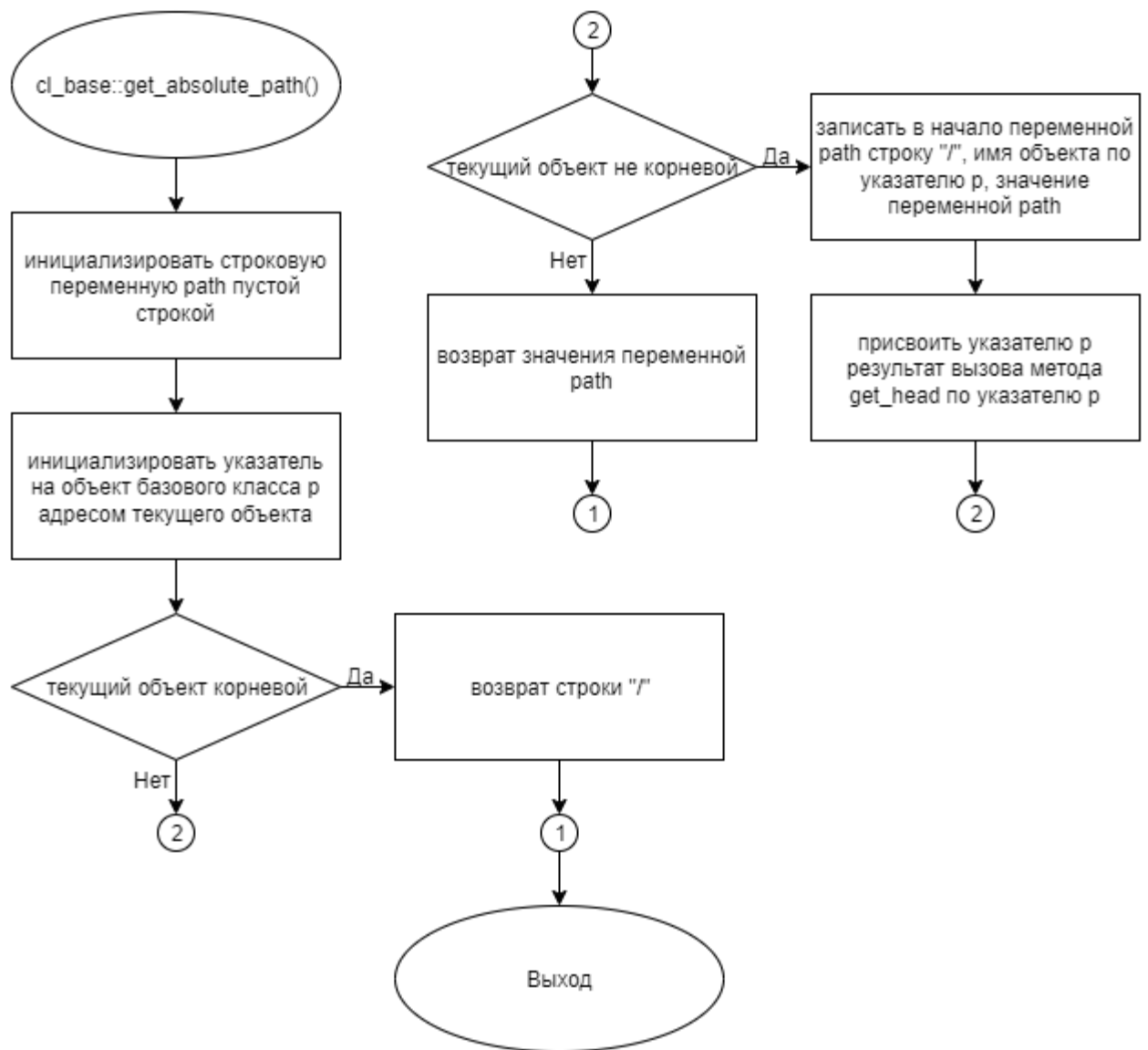


Рисунок 5 – Блок-схема алгоритма

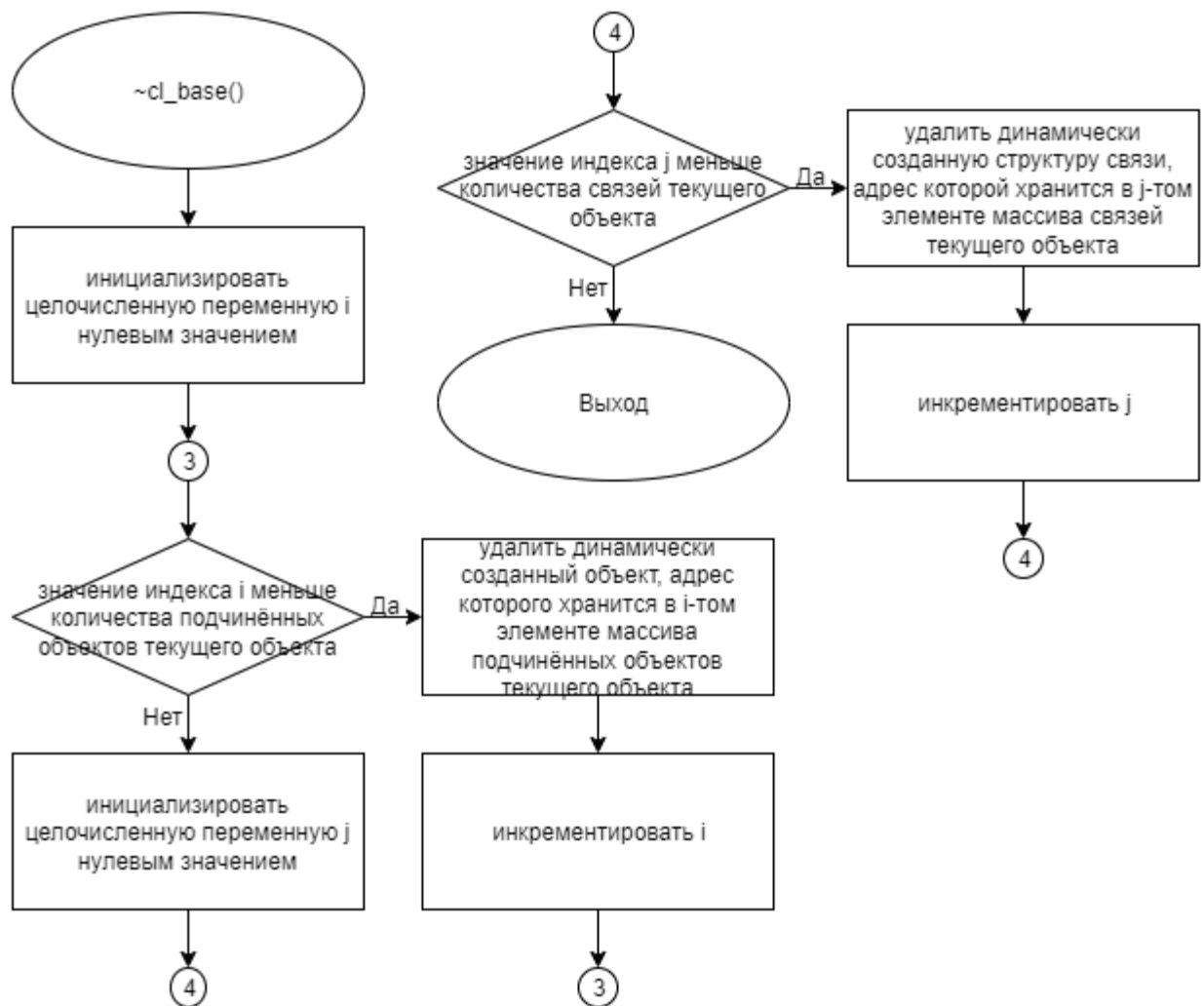


Рисунок 6 – Блок-схема алгоритма

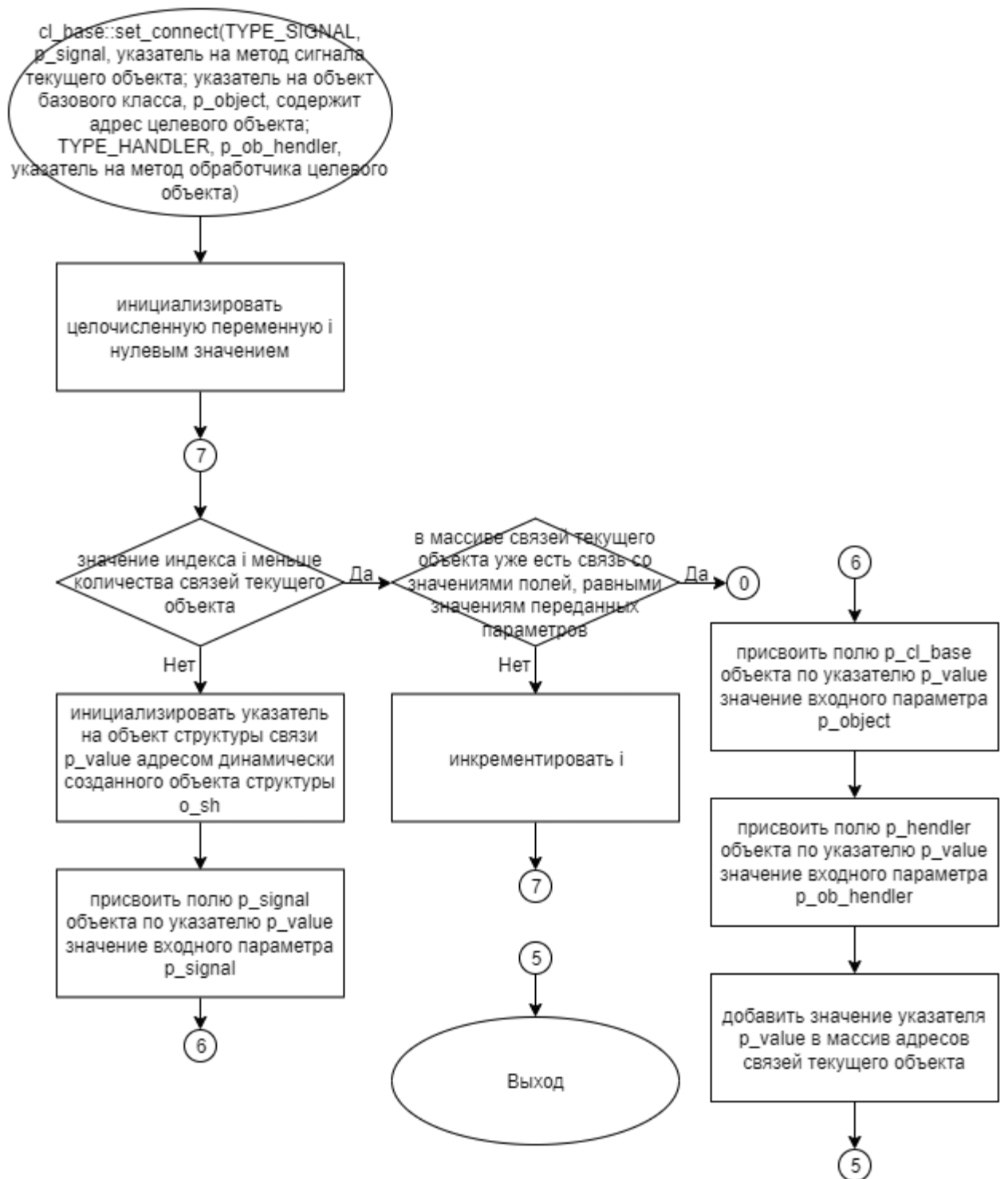


Рисунок 7 – Блок-схема алгоритма

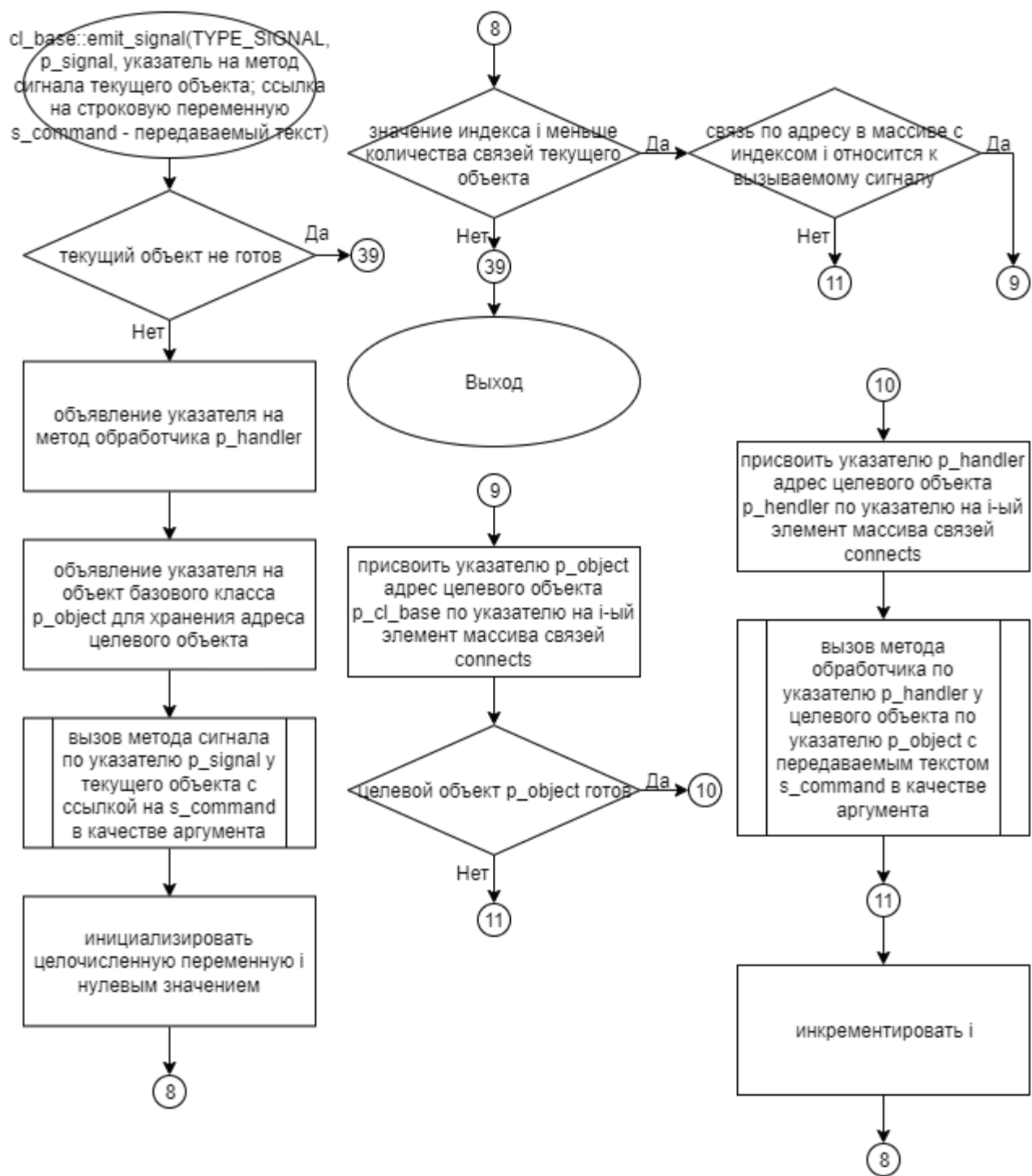


Рисунок 8 – Блок-схема алгоритма

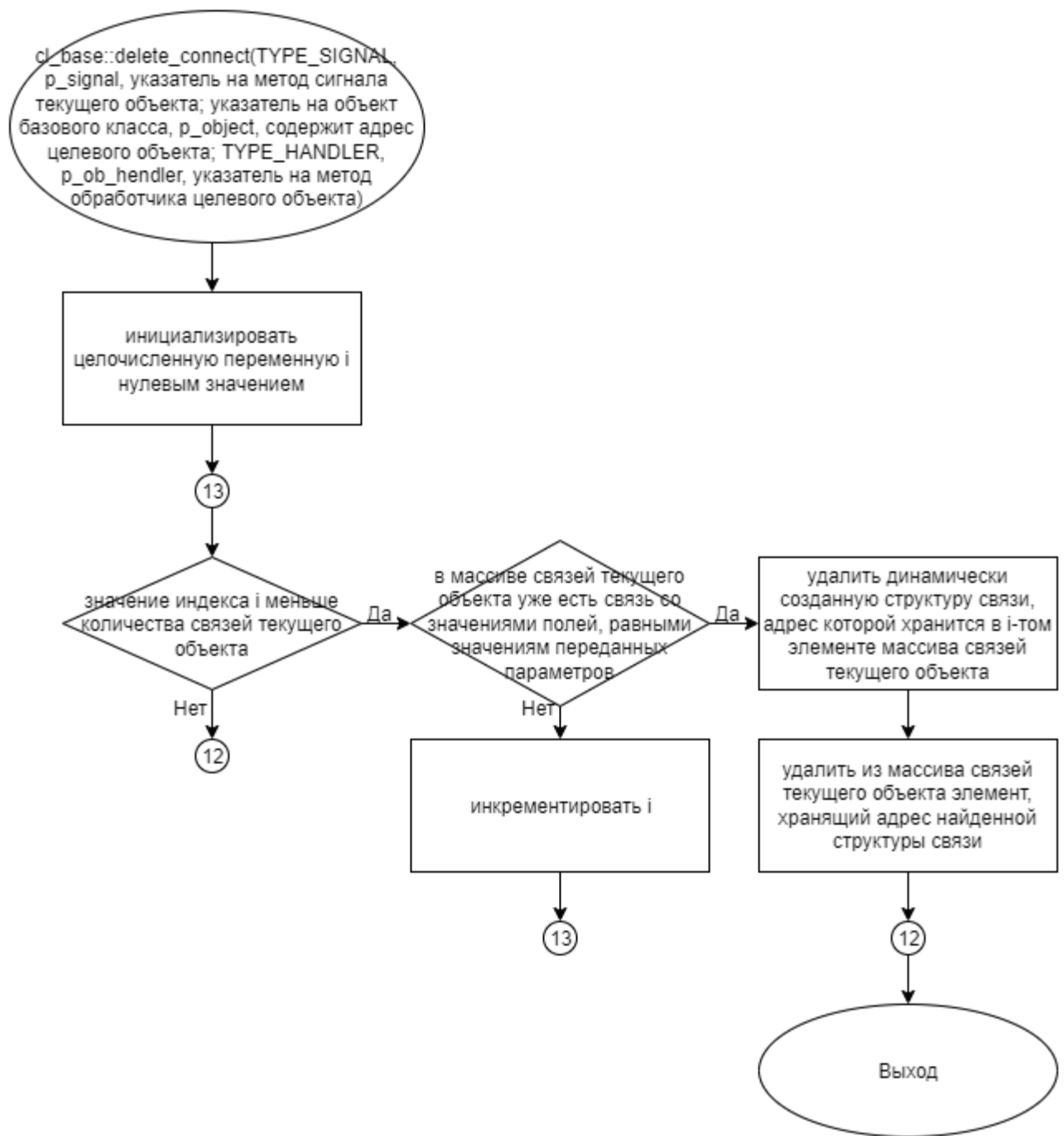


Рисунок 9 – Блок-схема алгоритма

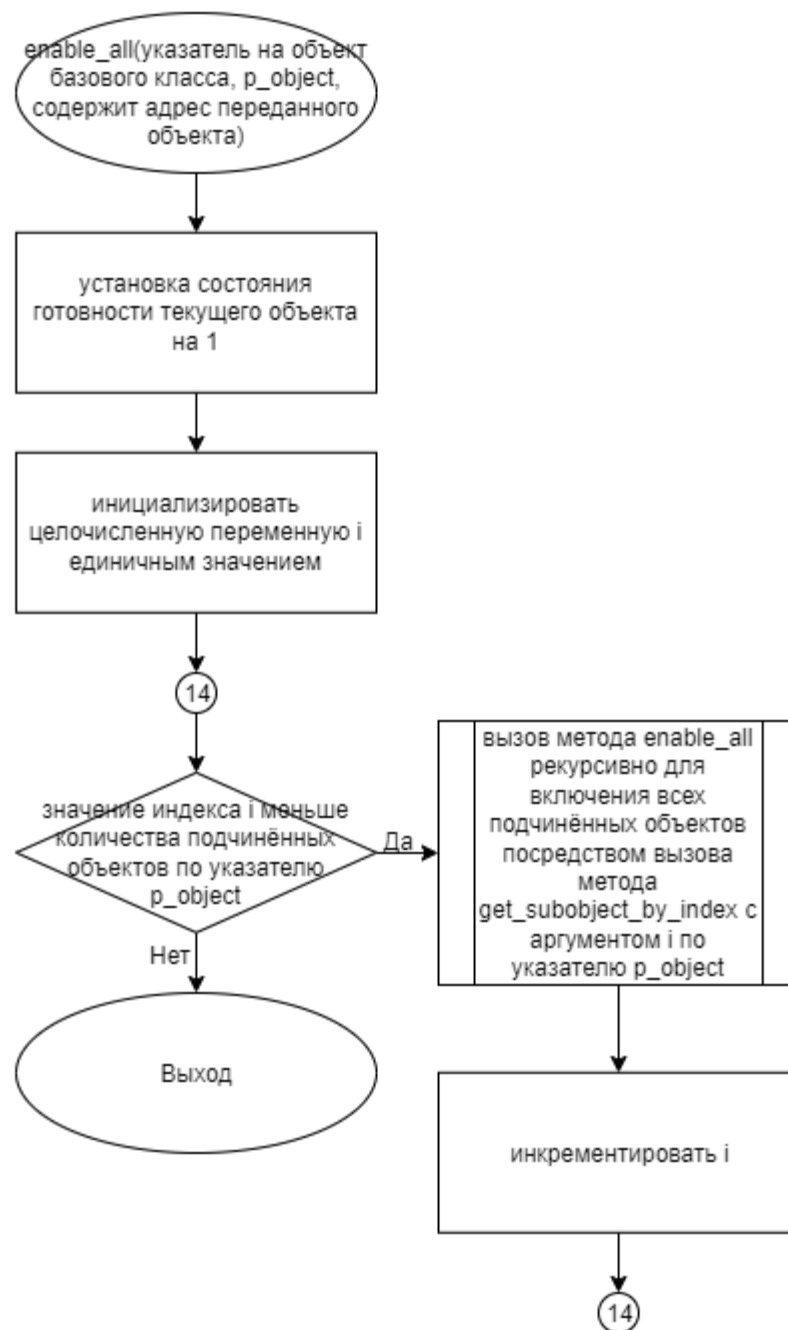


Рисунок 10 – Блок-схема алгоритма

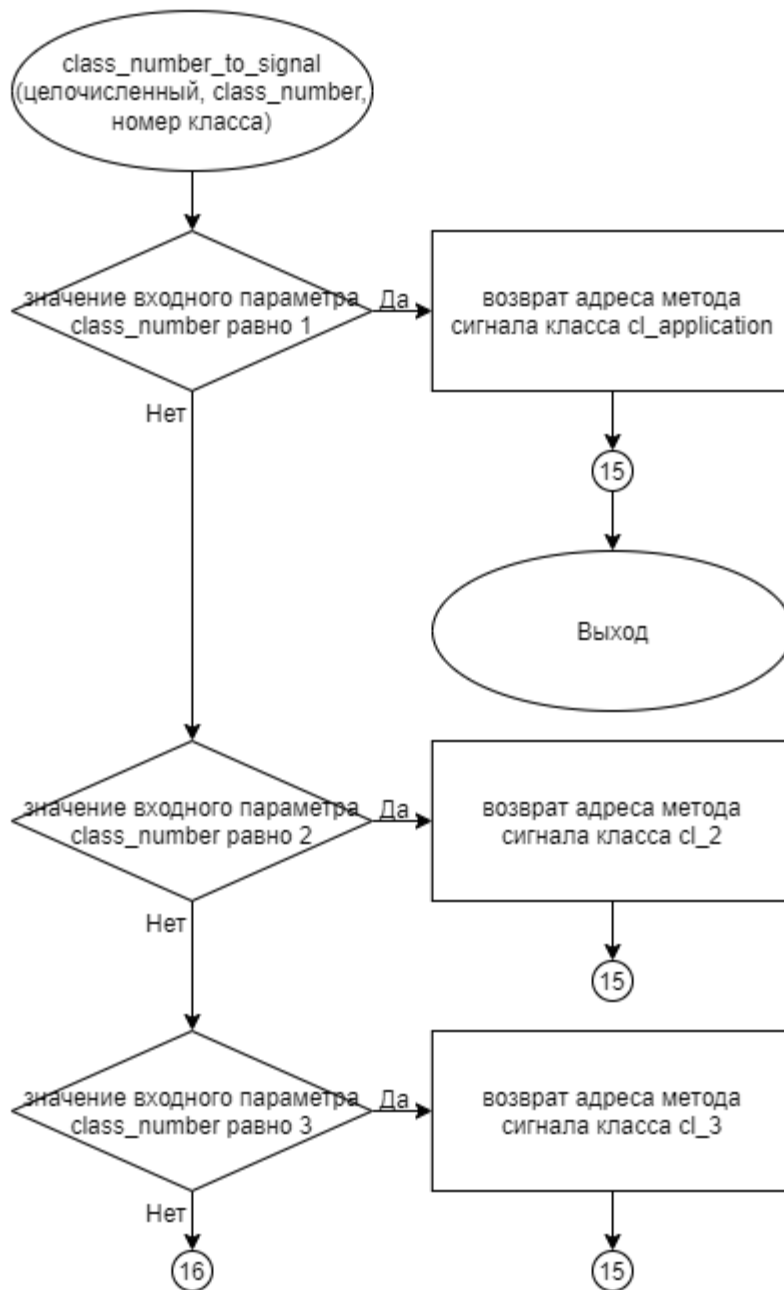


Рисунок 11 – Блок-схема алгоритма

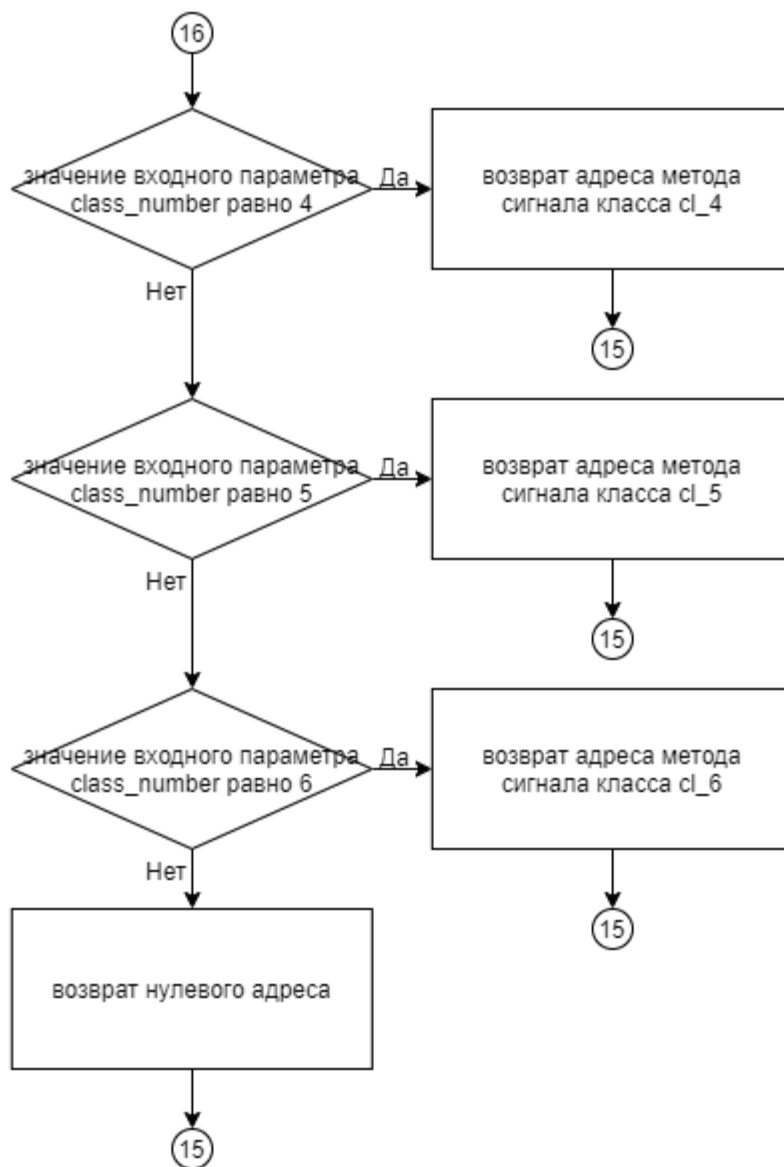


Рисунок 12 – Блок-схема алгоритма

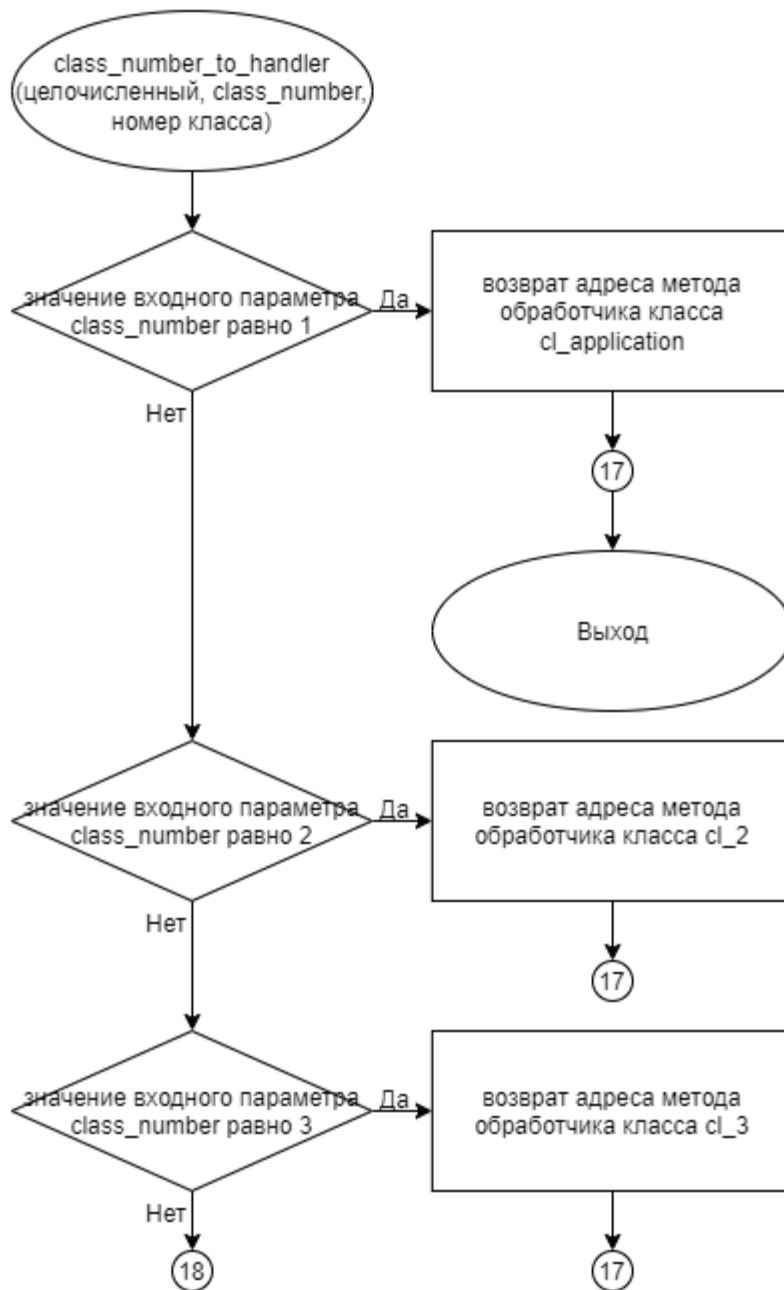


Рисунок 13 – Блок-схема алгоритма

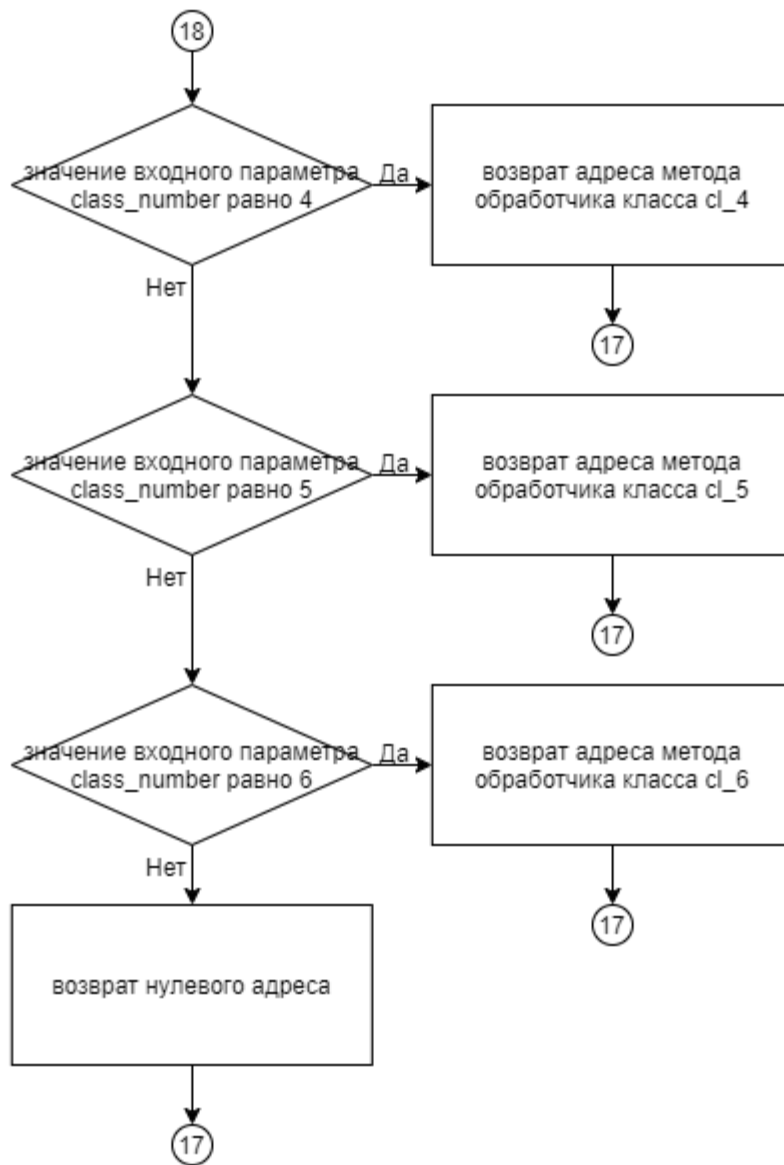


Рисунок 14 – Блок-схема алгоритма

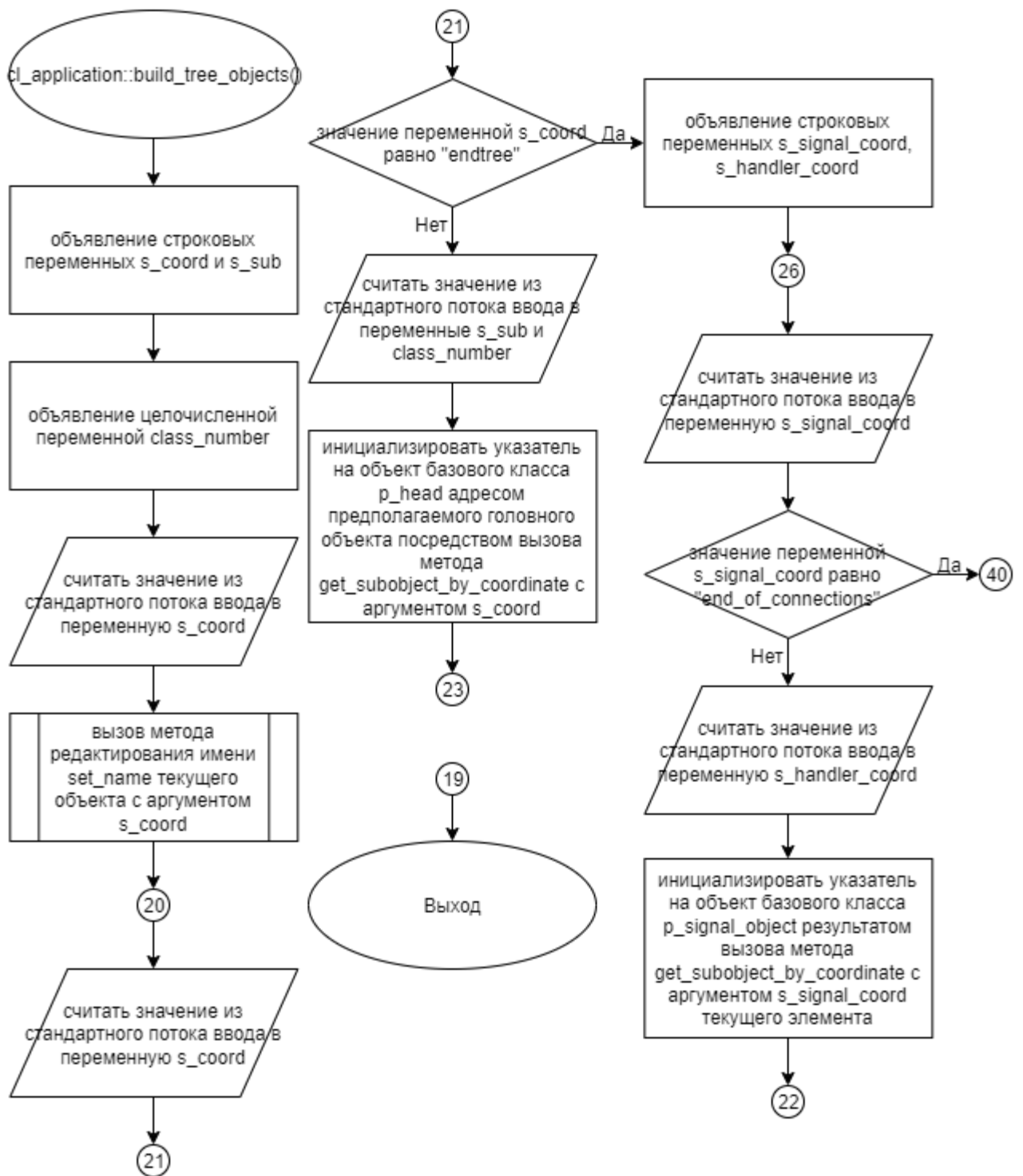


Рисунок 15 – Блок-схема алгоритма

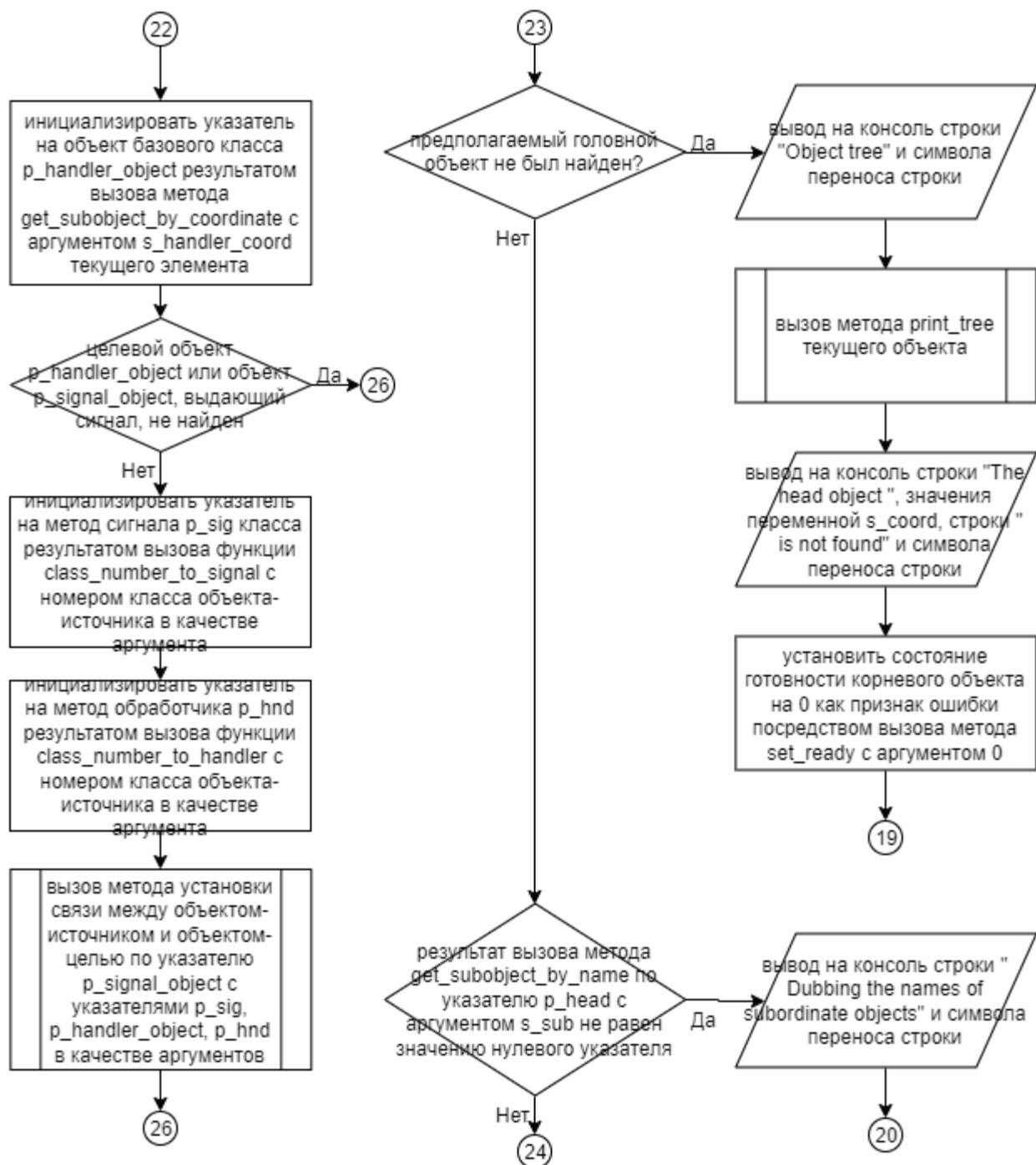


Рисунок 16 – Блок-схема алгоритма

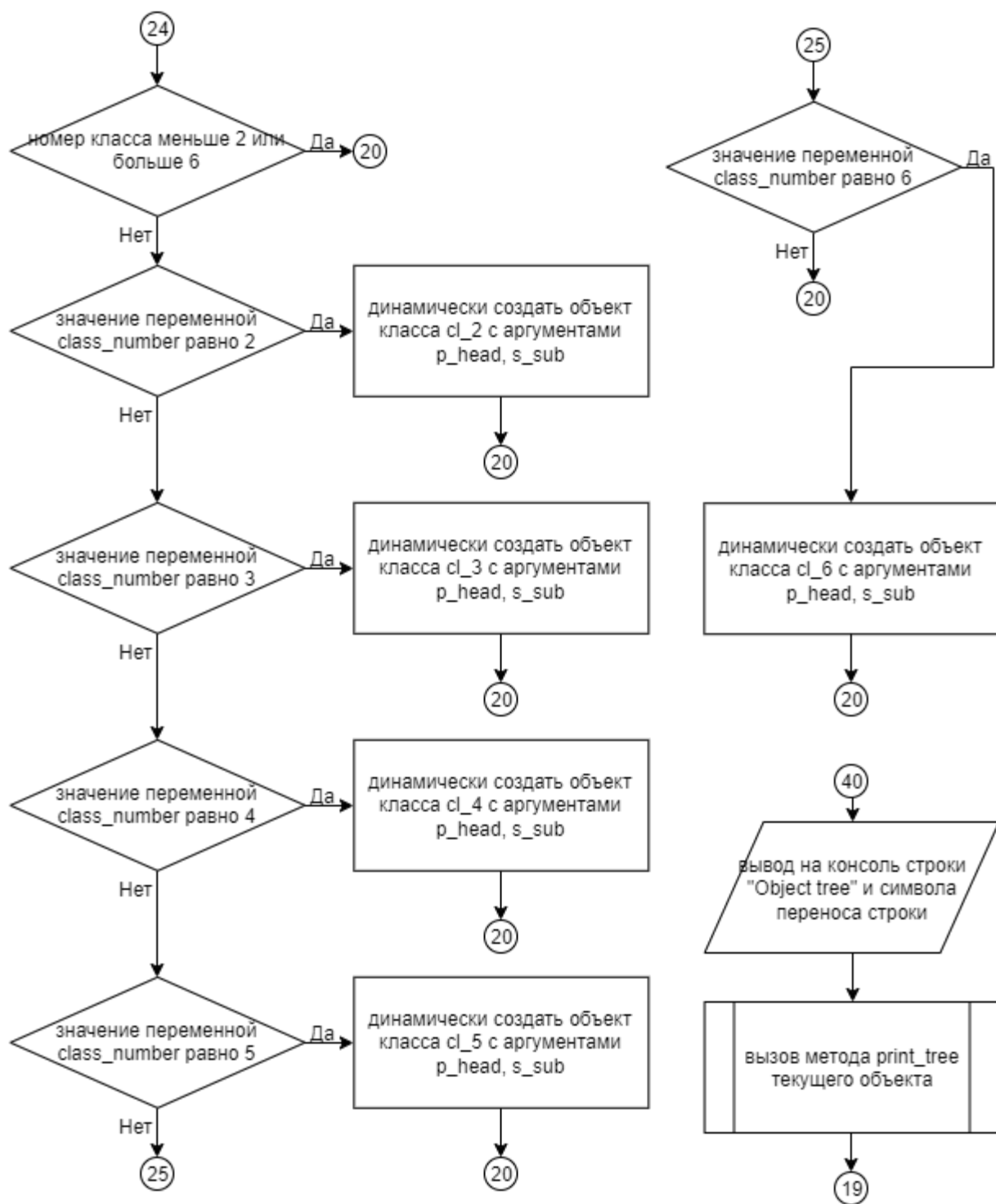


Рисунок 17 – Блок-схема алгоритма

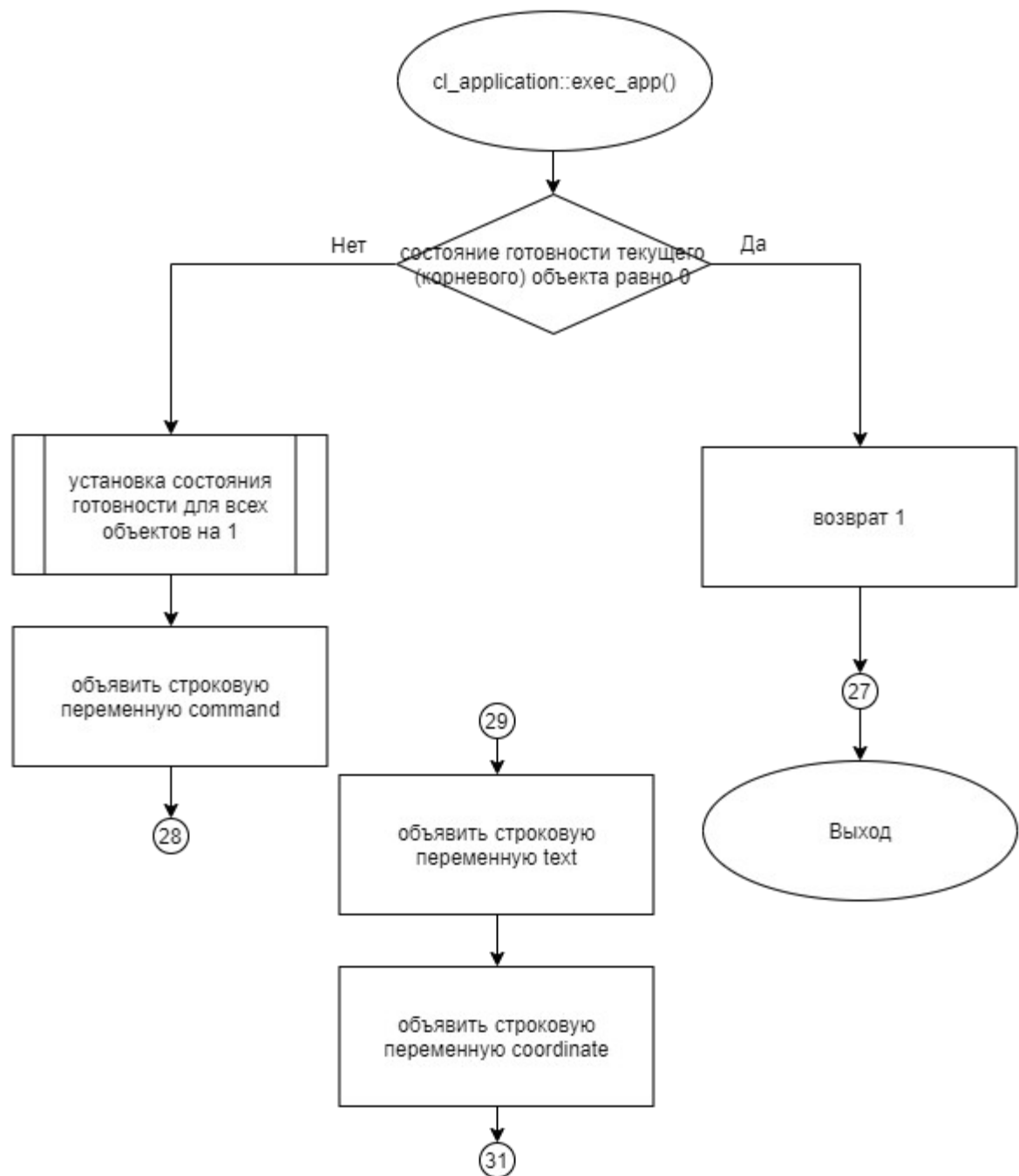


Рисунок 18 – Блок-схема алгоритма

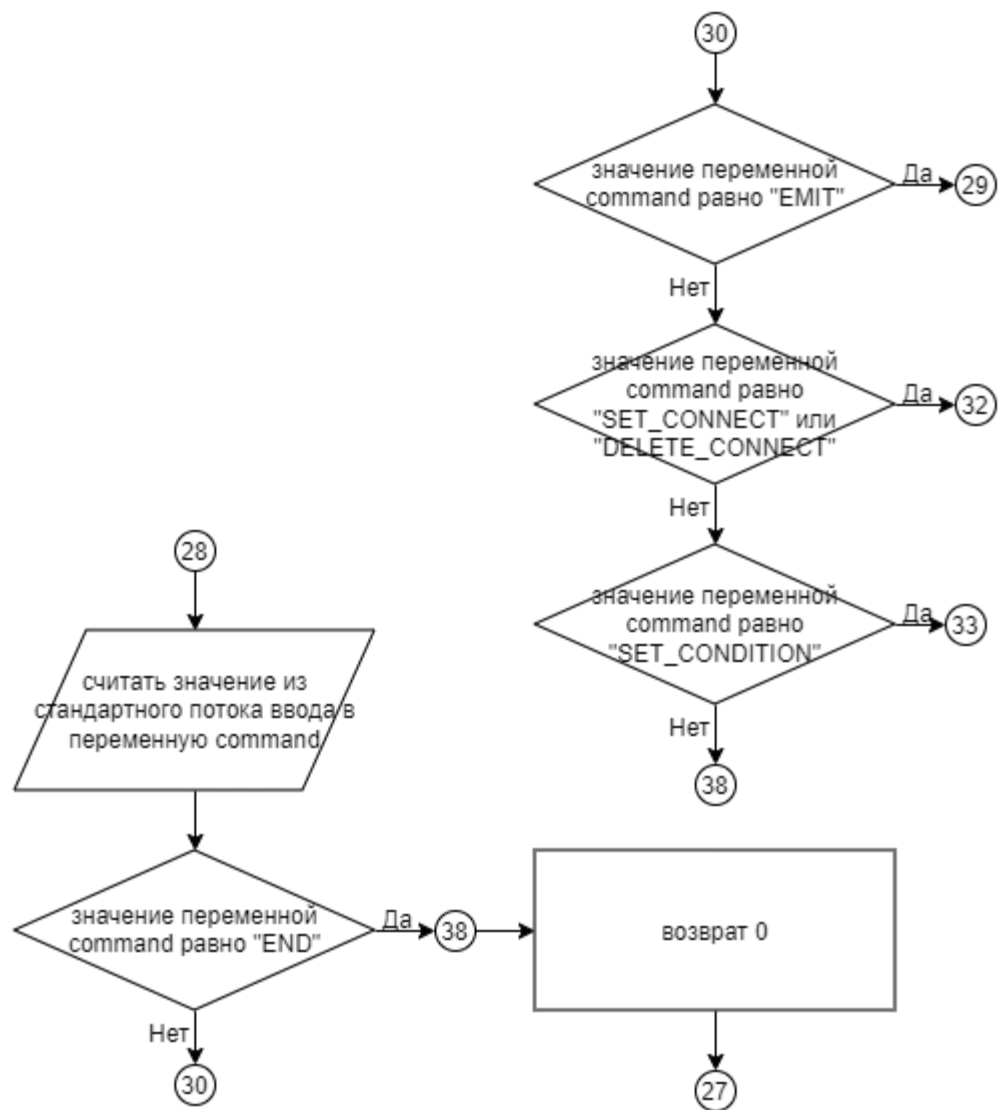


Рисунок 19 – Блок-схема алгоритма

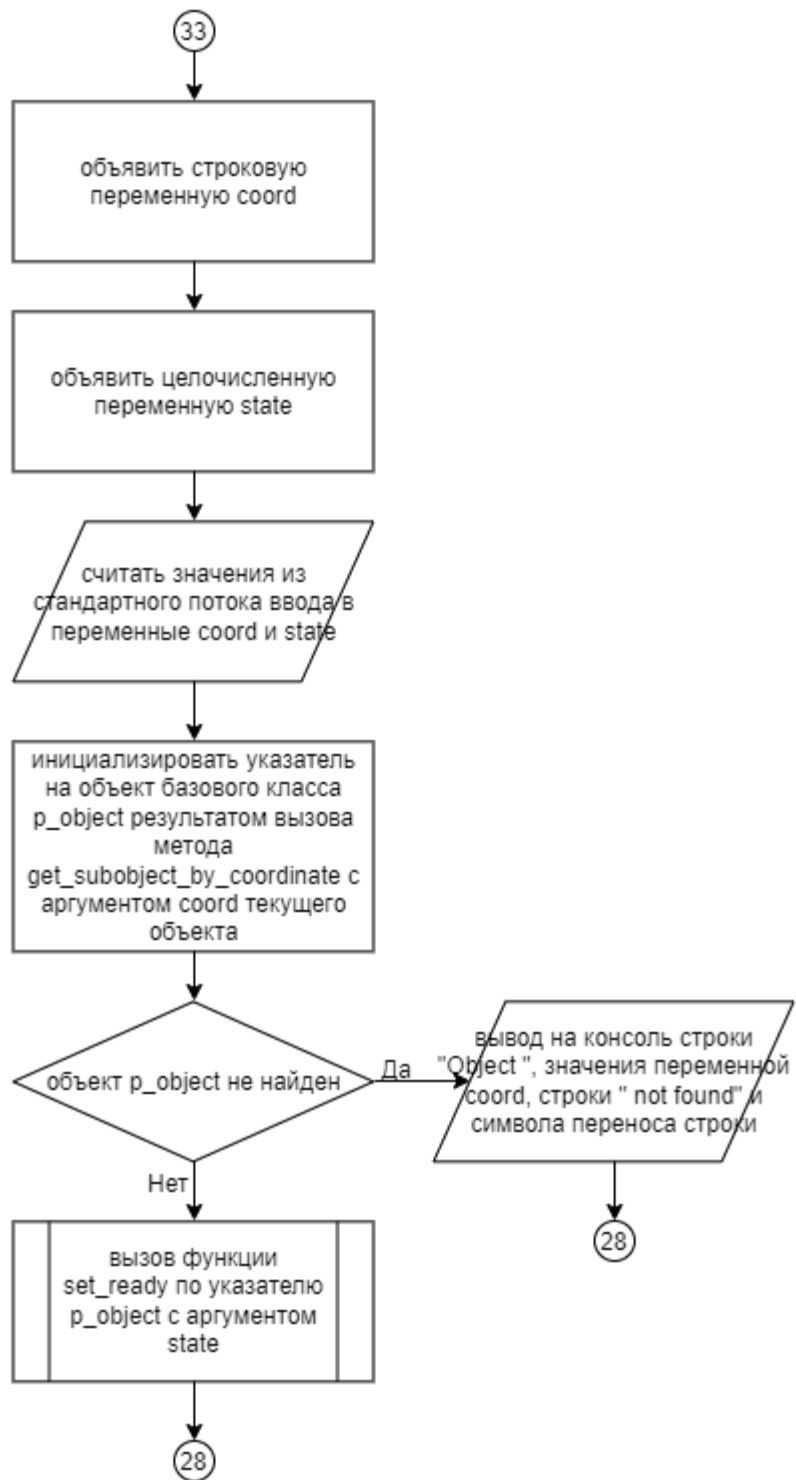


Рисунок 20 – Блок-схема алгоритма

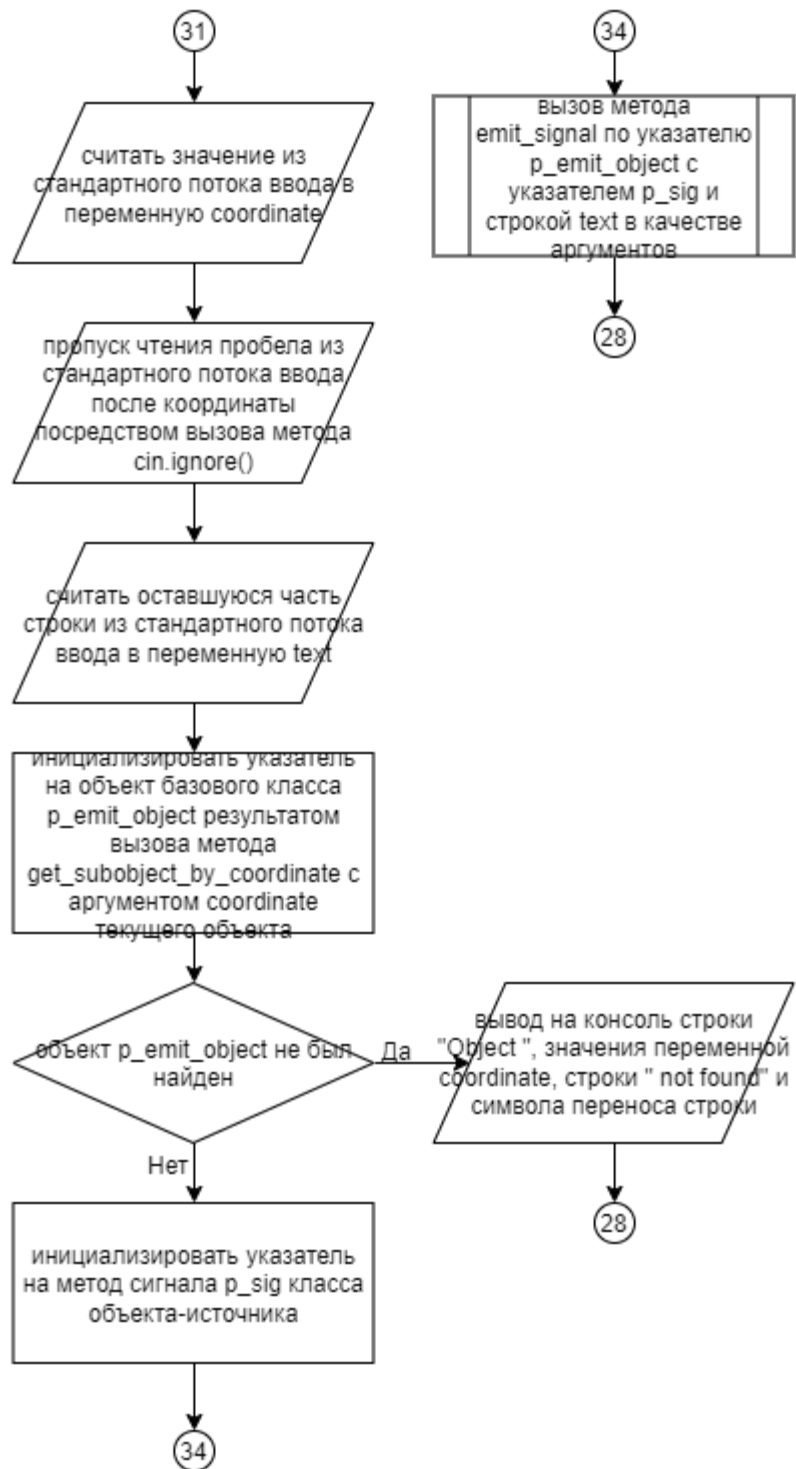


Рисунок 21 – Блок-схема алгоритма

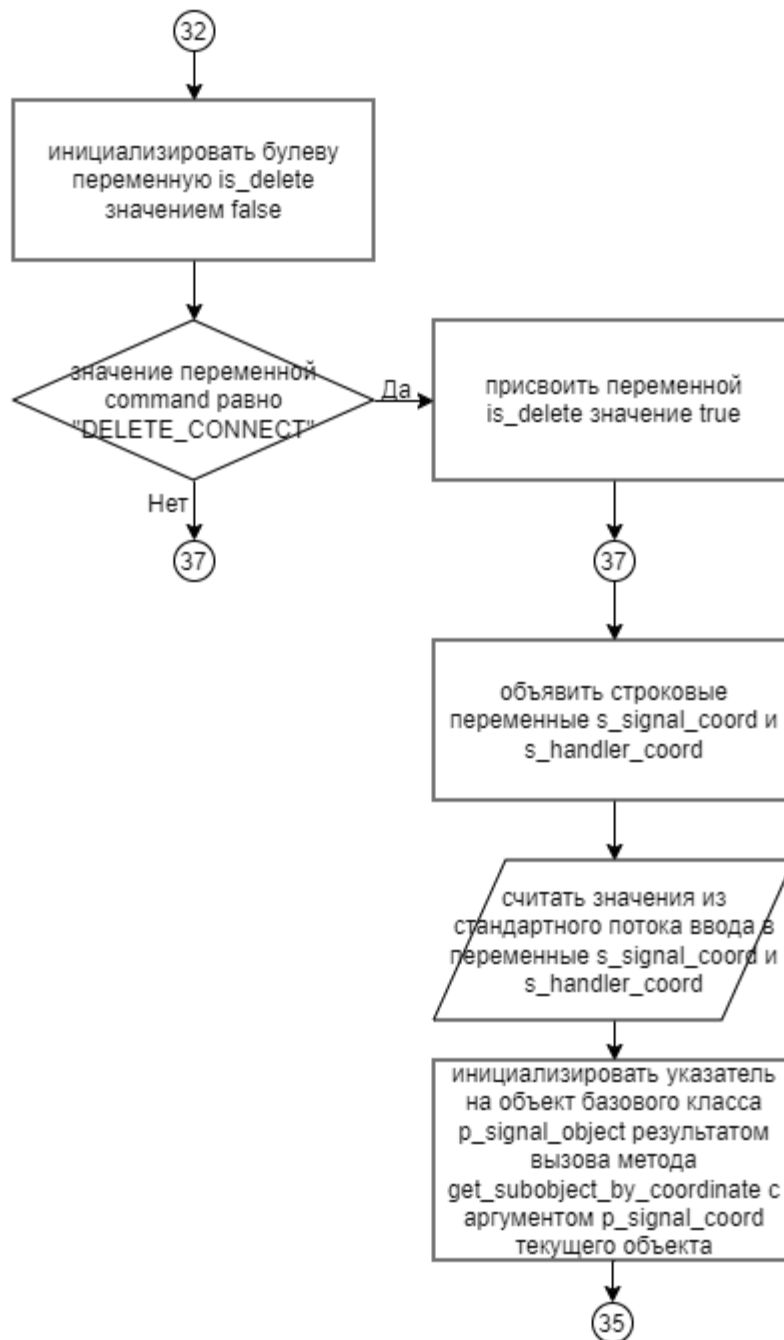


Рисунок 22 – Блок-схема алгоритма

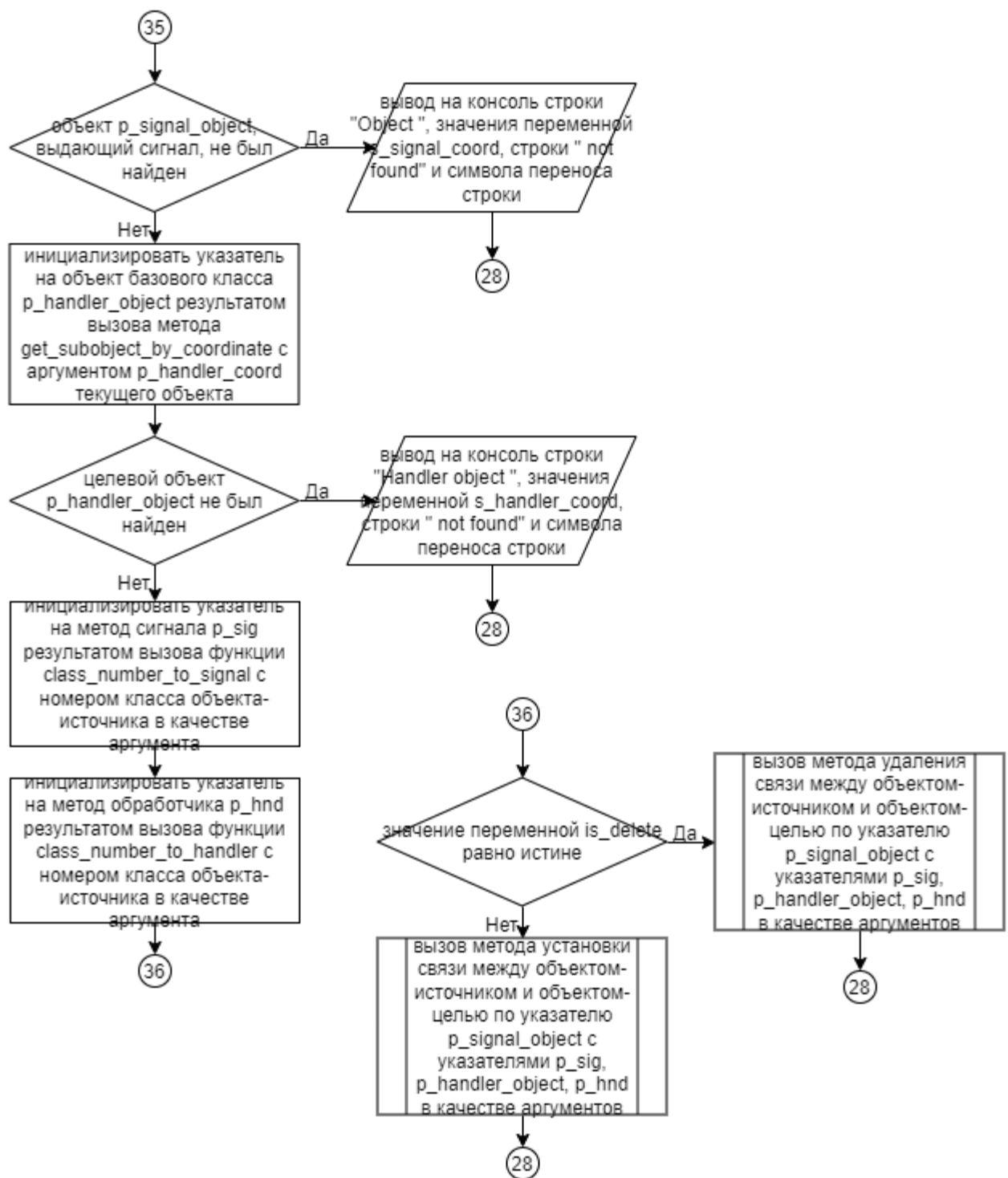


Рисунок 23 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_2.cpp

Листинг 1 – cl_2.cpp

```
#include "cl_2.h"
#include <iostream>
using namespace std;

cl_2 :: cl_2 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }
    // конструирует объект производного класса с указателем на головной
    // объект и наименованием путём вызова конструктора базового класса

int cl_2 :: get_class_number ( )
{
    return 2;
}

void cl_2 :: signal_cl_2 ( string & s_text )
    // метод сигнала класса cl_2
{
    cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
    // координаты объекта, выдающего сигнал
    s_text += " (class: 2)"; //
    // добавление номера класса к переданной строке
}

void cl_2 :: handler_cl_2 ( string s_text )
    // метод обработчика класса cl_2
{
    cout << "Signal to " << this -> get_absolute_path ( )
        << " Text:  " << s_text << endl;
    // вывод информации о получении сигнала
}
```

5.2 Файл cl_2.h

Листинг 2 – cl_2.h

```
#ifndef __CL_2__H
#define __CL_2__H

#include "cl_base.h"

class cl_2 : public cl_base
{
public:
    cl_2 ( cl_base * p_head_object, string s_object_name ); //
    параметризированный конструктор

    int get_class_number ( ); //      возвращает
    номер класса 2

    void signal_cl_2 ( string & s_text ); //      метод
    сигнала класса cl_2
    void handler_cl_2 ( string s_text ); //      метод
    обработчика класса cl_2
};

#endif
```

5.3 Файл cl_3.cpp

Листинг 3 – cl_3.cpp

```
#include "cl_3.h"
#include <iostream>
using namespace std;

cl_3 :: cl_3 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }
    // конструирует объект производного класса с указателем на головной
    объект и наименованием путём вызова конструктора базового класса

int cl_3 :: get_class_number ( )
{
    return 3;
}

void cl_3 :: signal_cl_3 ( string & s_text )
    // метод сигнала класса cl_3
{
    cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
```

```

    координаты объекта, выдающего сигнал
    s_text += " (class: 3)"; //
добавление номера класса к переданной строке
}

void cl_3 :: handler_cl_3 ( string s_text )
    // метод обработчика класса cl_3
{
    cout << "Signal to " << this -> get_absolute_path ( )
        << " Text:  " << s_text << endl;
    // вывод информации о получении сигнала
}

```

5.4 Файл cl_3.h

Листинг 4 – cl_3.h

```

#ifndef __CL_3__H
#define __CL_3__H

#include "cl_base.h"

class cl_3 : public cl_base
{
public:
    cl_3 ( cl_base * p_head_object, string s_object_name ); //
параметризированный конструктор

    int get_class_number ( ); // возвращает
номер класса 3

    void signal_cl_3 ( string & s_text ); // метод
сигнала класса cl_3
    void handler_cl_3 ( string s_text ); // метод
обработчика класса cl_3
};

#endif

```

5.5 Файл cl_4.cpp

Листинг 5 – cl_4.cpp

```

#include "cl_4.h"
#include <iostream>

```

```

using namespace std;

cl_4 :: cl_4 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }
    // конструирует объект производного класса с указателем на головной
    объект и наименованием путём вызова конструктора базового класса

int cl_4 :: get_class_number ( )
{
    return 4;
}

void cl_4 :: signal_cl_4 ( string & s_text )
    // метод сигнала класса cl_4
{
    cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
    координаты объекта, выдающего сигнал
    s_text += " (class: 4)"; //
    добавление номера класса к переданной строке
}

void cl_4 :: handler_cl_4 ( string s_text )
    // метод обработчика класса cl_4
{
    cout << "Signal to " << this -> get_absolute_path ( )
        << " Text: " << s_text << endl;
    // вывод информации о получении сигнала
}

```

5.6 Файл cl_4.h

Листинг 6 – cl_4.h

```

#ifndef __CL_4__H
#define __CL_4__H

#include "cl_base.h"

class cl_4 : public cl_base
{
public:
    cl_4 ( cl_base * p_head_object, string s_object_name ); //
    параметризованный конструктор

    int get_class_number ( ); // возвращает
    номер класса 4

    void signal_cl_4 ( string & s_text ); // метод
    сигнала класса cl_4
    void handler_cl_4 ( string s_text ); // метод

```

```
    обработчика класса cl_4
};

#endif
```

5.7 Файл cl_5.cpp

Листинг 7 – cl_5.cpp

```
#include "cl_5.h"
#include <iostream>
using namespace std;

cl_5 :: cl_5 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }
    // конструирует объект производного класса с указателем на головной
    объект и наименованием путём вызова конструктора базового класса

int cl_5 :: get_class_number ( )
{
    return 5;
}

void cl_5 :: signal_cl_5 ( string & s_text )
    // метод сигнала класса cl_5
{
    cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
    координаты объекта, выдающего сигнал
    s_text += " (class: 5)"; //
    добавление номера класса к переданной строке
}

void cl_5 :: handler_cl_5 ( string s_text )
    // метод обработчика класса cl_5
{
    cout << "Signal to " << this -> get_absolute_path ( )
        << " Text: " << s_text << endl;
    // вывод информации о получении сигнала
}
```

5.8 Файл cl_5.h

Листинг 8 – cl_5.h

```
#ifndef __CL_5__H
```

```

#define __CL_5__H

#include "cl_base.h"

class cl_5 : public cl_base
{
public:
    cl_5 ( cl_base * p_head_object, string s_object_name ); //
    параметризированный конструктор

    int get_class_number ( ); //      возвращает
    номер класса 5

    void signal_cl_5 ( string & s_text ); //      метод
    сигнала класса cl_5
    void handler_cl_5 ( string s_text ); //      метод
    обработчика класса cl_5
};

#endif

```

5.9 Файл cl_6.cpp

Листинг 9 – cl_6.cpp

```

#include "cl_6.h"
#include <iostream>
using namespace std;

cl_6 :: cl_6 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }
    // конструирует объект производного класса с указателем на головной
    объект и наименованием путём вызова конструктора базового класса

int cl_6 :: get_class_number ( )
{
    return 6;
}

void cl_6 :: signal_cl_6 ( string & s_text )
    // метод сигнала класса cl_6
{
    cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
    координаты объекта, выдающего сигнал
    s_text += " (class: 6)"; //
    добавление номера класса к переданной строке
}

void cl_6 :: handler_cl_6 ( string s_text )
    // метод обработчика класса cl_6

```

```

{
    cout << "Signal to " << this -> get_absolute_path ( )
          << " Text:  " << s_text << endl;
    // вывод информации о получении сигнала
}

```

5.10 Файл cl_6.h

Листинг 10 – cl_6.h

```

#ifndef __CL_6__H
#define __CL_6__H

#include "cl_base.h"

class cl_6 : public cl_base
{
public:
    cl_6 ( cl_base * p_head_object, string s_object_name ); //
    параметризированный конструктор

    int get_class_number ( ); //      возвращает
    номер класса 6

    void signal_cl_6 ( string & s_text ); //      метод
    сигнала класса cl_6
    void handler_cl_6 ( string s_text ); //      метод
    обработчика класса cl_6
};

#endif

```

5.11 Файл cl_application.cpp

Листинг 11 – cl_application.cpp

```

#include "cl_application.h"
#include "cl_2.h"
#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
#include <iostream>
#include <string>
#include <vector>

```

```

using namespace std;

cl_application :: cl_application ( cl_base * p_head_object, string
s_object_name )
: cl_base ( p_head_object, s_object_name ) { } // создаёт корневой
объект приложения, предварительно вызывая конструктор базового класса

// Возвращает указатель на сигнал по номеру класса объекта-источника
TYPE_SIGNAL class_number_to_signal ( int class_number )
{
    if ( class_number == 1 )
    {
        return SIGNAL_D( cl_application::signal_cl_1 );
    } else if ( class_number == 2 )
    {
        return SIGNAL_D( cl_2::signal_cl_2 );
    } else if ( class_number == 3 )
    {
        return SIGNAL_D( cl_3::signal_cl_3 );
    } else if ( class_number == 4 )
    {
        return SIGNAL_D( cl_4::signal_cl_4 );
    } else if ( class_number == 5 )
    {
        return SIGNAL_D( cl_5::signal_cl_5 );
    } else if ( class_number == 6 )
    {
        return SIGNAL_D( cl_6::signal_cl_6 );
    }
    return nullptr;
}

// Возвращает указатель на обработчик по номеру класса объекта-получателя
TYPE_HANDLER class_number_to_handler ( int class_number )
{
    if ( class_number == 1 )
    {
        return HENDLER_D( cl_application::handler_cl_1 );
    } else if ( class_number == 2 )
    {
        return HENDLER_D( cl_2::handler_cl_2 );
    } else if ( class_number == 3 )
    {
        return HENDLER_D( cl_3::handler_cl_3 );
    } else if ( class_number == 4 )
    {
        return HENDLER_D( cl_4::handler_cl_4 );
    } else if ( class_number == 5 )
    {
        return HENDLER_D( cl_5::handler_cl_5 );
    } else if ( class_number == 6 )
    {
        return HENDLER_D( cl_6::handler_cl_6 );
    }
    return nullptr;
}

```

```

}

int cl_application :: get_class_number ( )
{
    return 1;
}

void cl_application :: build_tree_objects ( )           // строит исходное
дерево иерархии объектов
{
    string s_coord, s_sub;                               // координата головного
объекта и имя подчинённого объектов                    // объявл. целочисленной
    int class_number;                                   // переменной class_number

    cin >> s_coord;                                     // считать значение из
стандартного потока ввода в переменную s_coord
    this -> set_name ( s_coord );                       // устанавливаем имя
текущего объекта значением s_head вызовом метода set_name

    while ( true )                                     // начало бесконечного цикла
    {
        cin >> s_coord;                                 // считать значение из
стандартного потока ввода в переменную s_coord

        if ( s_coord == "endtree" )                   // значение переменной
s_coord равно "endtree"
        {
            break;                                     // прервать выполнение цикла
- завершение построения дерева
        }

        cin >> s_sub >> class_number;                   // считать значение из
стандартного потока ввода в переменные s_sub и class_number

        cl_base * p_head = this -> get_subobject_by_coordinate ( s_coord );//
поиск головного по координате
        if ( p_head == nullptr )                     // головной объект не найден
        {
            cout << "Object tree" << endl;
            this -> print_tree ( );
            cout << "The head object " << s_coord << " is not found" << endl;
            this -> set_ready ( 0 );                 // устанавливаем состояние
готовности корневого объекта на -123 как признак ошибки
            return;
        }

        if ( p_head -> get_subobject_by_name ( s_sub ) != nullptr ) //
проверка дуближа
        {
            // на дереве уже есть объект с
именем создаваемого
            cout << s_coord << "          Dubbing the names of subordinate objects"
<< endl;
            continue;                                // обнаружен дуближ имени на
дереве - объект не создаётся, переход к следующей итерации
        }
    }
}

```

```

    }

    if ( class_number < 2 || class_number > 6 )      // номер класса введён
некорректно ( допустимые значения от 2 до 6)
    {
        continue;                                // номер задан некорректно
    }

    if ( class_number == 2 )                        // создание объекта по
номеру класса
    {
        new cl_2 ( p_head, s_sub );                // динамически создать
объект класса cl_2 с введёнными значениями в качестве аргументов
    } else if ( class_number == 3 )
    {
        new cl_3 ( p_head, s_sub );                // динамически создать
объект класса cl_3 с введёнными значениями в качестве аргументов
    } else if ( class_number == 4 )
    {
        new cl_4 ( p_head, s_sub );                // динамически создать
объект класса cl_4 с введёнными значениями в качестве аргументов
    } else if ( class_number == 5 )
    {
        new cl_5 ( p_head, s_sub );                // динамически создать
объект класса cl_5 с введёнными значениями в качестве аргументов
    } else if ( class_number == 6 )
    {
        new cl_6 ( p_head, s_sub );                // динамически создать
объект класса cl_6 с введёнными значениями в качестве аргументов
    }
}

string s_signal_coord, s_handler_coord;            // координаты объекта
выдающего сигнал и целевого объекта

while ( true )                                    // начало цикла ввода связей
сигнал-обработчик
{
    cin >> s_signal_coord;                        // считать координату
объекта выдающего сигнал

    if ( s_signal_coord == "end_of_connections" ) // признак завершения
ввода связей
    {
        break;                                    // прервать цикл ввода
связей
    }

    cin >> s_handler_coord;                        // считать координату
целевого объекта

    cl_base * p_signal_object = this -> get_subobject_by_coordinate
( s_signal_coord ); // поиск объекта выдающего сигнал
    cl_base * p_handler_object = this -> get_subobject_by_coordinate
( s_handler_coord ); // поиск целевого объекта

```

```

        if ( p_signal_object == nullptr || p_handler_object == nullptr ) //
        {
            continue; // переход к следующей
            итерации
        }

        TYPE_SIGNAL p_sig = class_number_to_signal ( p_signal_object ->
        get_class_number ( ) );
        TYPE_HANDLER p_hnd = class_number_to_handler ( p_handler_object ->
        get_class_number ( ) );

        p_signal_object -> set_connect ( p_sig, p_handler_object, p_hnd );
    }

    cout << "Object tree" << endl; // вывод заголовка
    this -> print_tree ( ); // отображение
    иерархии
}

void enable_all ( cl_base * p_object )
{
    p_object -> set_ready ( 1 );
    // установка состояния готовности текущего объекта на 1

    for ( int i = 1; i <= p_object -> get_subordinate_count ( ); i++ ) //
    {
        enable_all ( p_object -> get_subobject_by_index ( i ) ); //
        // вызов метода enable_all рекурсивно для включения всех подчинённых объектов
    }
}

int cl_application :: exec_app ( ) // запускает
систему
{
    if ( this -> status == 0 )
    {
        return 1;
    }

    enable_all ( this ); // установка состояния
    готовности для всех объектов на 1
    string command; // строка для хранения
    команды

    while ( true ) // начало бесконечного цикла
    {
        // обработка команд
        cin >> command; // считать команду

        if ( command == "END" ) // команда END
        {
            break; // завершить обработку
        }
    }
}

```

```

команд
}
else if ( command == "EMIT" ) // команда EMIT -
выдача сигнала от объекта
{
    string text; // строка для хранения текста
сообщения
    string coordinate; // строка для хранения
координаты объекта
    cin >> coordinate; // считать координату
объекта
    cin.ignore ( ); // пропуск пробела после
координаты
    getline ( cin, text ); // считать оставшуюся часть
строки как текст

    cl_base * p_emit_object = this -> get_subobject_by_coordinate
( coordinate ); // поиск объекта по координате

    if ( p_emit_object == nullptr ) // объект не найден
    {
        cout << "Object " << coordinate << " not found" << endl; // вывод
сообщения об ошибке
        continue; // переход к следующей итерации
    }

    TYPE_SIGNAL p_sig = class_number_to_signal ( p_emit_object ->
get_class_number ( ) );
    p_emit_object -> emit_signal ( p_sig, text );
}
else if ( command == "SET_CONNECT" || command == "DELETE_CONNECT" )
// команда установки или удаления связи
{
    bool is_delete = false; // флаг: 0 -
установить связь, 1 - удалить связь
    if ( command == "DELETE_CONNECT" )
    {
        is_delete = true;
    }

    string s_signal_coord, s_handler_coord; // координаты объектов
    cin >> s_signal_coord >> s_handler_coord; // считать координаты

    cl_base * p_signal_object = this -> get_subobject_by_coordinate
( s_signal_coord ); // поиск объекта выдающего сигнал

    if ( p_signal_object == nullptr ) // объект выдающий сигнал не
найден
    {
        cout << "Object " << s_signal_coord << " not found" << endl; //
вывод сообщения об ошибке
        continue; // переход к следующей итерации
    }

    cl_base * p_handler_object = this -> get_subobject_by_coordinate

```

```

( s_handler_coord );// поиск целевого объекта

        if ( p_handler_object == nullptr )           // целевой объект не
найден
        {
            cout << "Handler object " << s_handler_coord << " not found" <<
endl;// вывод сообщения об ошибке
            continue;                                // переход к следующей итерации
        }

        TYPE_SIGNAL p_sig = class_number_to_signal ( p_signal_object ->
get_class_number ( ) );
        TYPE_HANDLER p_hnd = class_number_to_handler ( p_handler_object ->
get_class_number ( ) );

        if ( is_delete )                             // удаление связи по номеру класса
объекта-источника сигнала
        {
            p_signal_object -> delete_connect ( p_sig, p_handler_object,
p_hnd );
        }
        else                                         // установка связи по номеру
класса объекта-источника сигнала
        {
            p_signal_object -> set_connect ( p_sig, p_handler_object,
p_hnd );
        }
    }
    else if ( command == "SET_CONDITION" )           // команда SET_CONDITION -
установление состояния объекта
    {
        string coord;                               // координата объекта
        int state;                                   // состояние готовности
        cin >> coord >> state;                       // считать координату и
состояние

        cl_base * p_object = this -> get_subobject_by_coordinate
( coord );// поиск объекта по координате

        if ( p_object == nullptr )                 // объект не найден
        {
            cout << "Object " << coord << " not found" << endl;// вывод
сообщения об ошибке
            continue;                                // переход к следующей итерации
        }

        p_object -> set_ready ( state );           // установка состояния
готовности
    }
}

return 0;                                           // завершить работу метода
}

void cl_application :: signal_cl_1 ( string & s_text )// метод сигнала

```

```

    класса cl_application (класс 1)
    {
        cout << "Signal from " << this -> get_absolute_path ( ) << endl; // вывод
        координаты объекта, выдающего сигнал
        s_text += " (class: 1)"; // добавление номера класса
        к переданной строке
    }

    void cl_application :: handler_cl_1 ( string s_text ) // метод обработчика
    класса cl_application (класс 1)
    {
        cout << "Signal to " << this -> get_absolute_path ( )
        << " Text:  " << s_text << endl; // вывод информации о получении сигнала
    }

```

5.12 Файл cl_application.h

Листинг 12 – cl_application.h

```

#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"

class cl_application : public cl_base
{
public:
    cl_application ( cl_base * p_head_object, std :: string s_object_name =
    "Application" ); // параметризованный конструктор
    void build_tree_objects ( );
    // построение исходного дерева иерархии объектов на основе входных данных
    int exes_app ( );
    // запуск системы, обработка команд над объектами

    int get_class_number ( );
        // возвращает номер класса 1

    void signal_cl_1 ( std :: string & s_text );
        // метод сигнала класса cl_application (класс 1)
    void handler_cl_1 ( std :: string s_text );
        // метод обработчика класса cl_application (класс 1)
};

#endif

```

5.13 Файл cl_base.cpp

Листинг 13 – cl_base.cpp

```
#include "cl_base.h"
#include <iostream>
#include <sstream>
using namespace std;

cl_base :: cl_base ( cl_base * p_head_object, string s_object_name )    //
параметризированный конструктор
{
    this -> s_object_name = s_object_name;
    // хранение наименования объекта
    this -> p_head_object = p_head_object;
    // хранение адреса на головной объект

    if ( p_head_object ) {
        // есть ли головной объект?
        p_head_object -> subordinate_objects.push_back ( this );
        // добавляем адрес текущего объекта в список адресов подчинённых объектов
        // головного объекта
    }
}

cl_base :: ~cl_base ( )
// деструктор
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
    {
        // значение индекса i меньше размера вектора подчинённых объектов?
        delete subordinate_objects [ i ];
        // удалить динамически созданный объект, адрес которого хранится в i-
        // том элементе массива подчинённых объектов текущего объекта
    }

    for ( int j = 0; j < (int) connects.size ( ); j++ )
    {
        // значение индекса j меньше количества связей текущего объекта
        delete connects [ j ];
        // удалить динамически созданную структуру связи, адрес которой хранится
        // в j-том элементе массива связей текущего объекта
    }
}

bool cl_base :: set_name ( string new_name )
// редактирует имя объекта с проверкой отсутствия дуближа среди
// подчинённых объектов у головного
{
    if ( this -> p_head_object != nullptr )
        // есть ли головной объект?
    {
        for ( int i = 0; i < p_head_object -> get_subordinate_count ( ); i++ )
        {
            //
```

```

значение индекса i меньше количества подчинённых объектов головного объекта
        if ( p_head_object -> subordinate_objects [ i ] -> get_name ( ) ==
new_name )
        {
            //
            проверяем уникальность имени
            return false;
            //
            есть другой подчинённый с таким же именем
        }
    }

    this -> s_object_name = new_name;
    // установка нового имени
    return true;
    // имя успешно изменено
}

string cl_base :: get_name ( )
    // возвращает наименование объекта
{
    return this -> s_object_name;
    // текущее наименование объекта
}

cl_base * cl_base :: get_head()
    // возвращает адрес головного объекта текущего объекта
{
    return this -> p_head_object;
    // адрес головного объекта текущего объекта
}

int cl_base :: get_subordinate_count ( )
    // возвращает количество подчинённых объектов текущего объекта
{
    return (int) this -> subordinate_objects.size ( );
    // текущее количество подчинённых объектов
}

cl_base * cl_base :: get_subobject_by_name ( string name )
    // возвращает адрес подчинённого объекта по его имени (если не найден -
возвращает nullptr)
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
        //
        значение индекса i меньше количества элементов в списке подчинённых объектов
        текущего объекта
        {
            if ( subordinate_objects [ i ] -> s_object_name == name )
                //
                имя текущего подчинённого объекта совпадает со значением входного параметра
                name
                return subordinate_objects [ i ];
            // вернуть значение i-того указателя из массива подчинённых объектов
        }

    return nullptr;
}

```

```

        // объект не найден
    }

    cl_base * cl_base :: get_subobject_by_index ( int index )
        // возвращает адрес подчинённого объекта по его индексу (от 1 до
        количества подчинённых)
    {
        if ( index < 1 || index > this -> get_subordinate_count ( ) ) //
        проверка на корректность индекса
        {
            return nullptr;
            // объект не найден
        }

        return subordinate_objects [ index - 1 ];
        // такой подчинённый существует
    }

    cl_base * cl_base :: get_subobject_on_branch ( string name ) //
    метод поиска объекта на ветке дерева иерархии от текущего по имени
    {
        vector < cl_base * > queue;
        // массив для хранения очереди обхода
        queue.push_back ( this );
        // добавить адрес текущего объекта в очередь
        cl_base * p_found = nullptr;
        // инициализировать указатель на объект базового класса p_found значением
        нулевого указателя

        for ( int i = 0; i < queue.size ( ); i++)
            // значения индекса i меньше количества элементов в очереди queue
        {
            if ( queue [ i ] -> get_name ( ) == name )
                // имя i-го элемента из queue совпадает с искомым
            {
                if ( p_found != nullptr )
                    // значение указателя p_found не нулевое
                {
                    return nullptr;
                    // вернуть значение нулевого указателя
                }
                p_found = queue [ i ];
                // присвоить переменной p_found значение i-го элемента из queue
            }

            for ( int j = 1; j <= queue [ i ] -> get_subordinate_count ( ); j++ )//
            значение индекса j меньше количества подчинённых объектов у i-го элемента из
            queue
            {
                queue.push_back ( queue [ i ] -> get_subobject_by_index ( j ) );//
                добавить в очередь адрес найденного элемента по индексу j по указателю на i-
                ый элемент массива queue
            }
        }

        return p_found;
    }

```

```

        // вернуть адрес найденного объекта p_found
    }

    cl_base * cl_base :: get_subobject_on_tree ( string name )
    // метод поиска объекта на всём дереве иерархии по имени
    {
        cl_base * root = this;
        // иниц. указатель root значением адреса текущего объекта

        while ( root -> get_head() != nullptr )
            // у объекта, на который указывает root, есть головной объект
            {
                root = root -> get_head ( );
                // присвоить указателю root адрес его головного объекта
            }

        return root -> get_subobject_on_branch ( name );
        // вернуть результат вызова метода get_url_on_branch по указателю root на
        // корневой объект root
    }

    void cl_base :: print_tree ( int depth )
    // метод вывода иерархии объектов (дерева или ветки) от текущего
    // объекта
    {
        for ( int i = 0; i < depth; i++ )
            // вывод отступа: по 4 пробела на каждый уровень вложенности
            {
                cout << "    ";
                // вывод на консоль 4 пробелов
            }

        cout << this -> get_name ( ) << endl;
        // вывод имени текущего объекта и переход на новую строку
        for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )
            {
                subordinate_objects [ j ] -> print_tree ( depth + 1 );
                // вызов метода print_tree рекурсивно по указателю на i-тый элемент массива
                // подчинённых объектов с глубиной отступа +1 в качестве аргумента
            }
    }

    void cl_base :: print_ready ( int depth )
    // метод вывода иерархии объектов и состояний их готовности от текущего
    // объекта
    {
        for ( int i = 0; i < depth; i++ )
            // значение переменной i меньше значения входного параметра depth
            {
                cout << "    ";
                // вывод на консоль 4 пробелов
            }

        cout << this -> get_name ( );
        // вывод на консоль имени текущего объекта
    }

```

```

    if ( this -> status != 0 )
        // значение поля status текущего объекта не равно нулю
    {
        cout << " is ready" << endl;
        // объект готов
    }
    else
    {
        cout << " is not ready" << endl;
        // объект не готов
    }

    for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )           //
        // значение индекса j меньше размера массива подчинённых объектов
    {
        subordinate_objects [ j ] -> print_ready ( depth + 1 );
        // вызов метода print_ready рекурсивно по указателю на i-тый элемент
        // массива подчинённых объектов с глубиной отступа +1 в качестве аргумента
    }
}

void cl_base :: set_ready ( int state )
{
    if ( state != 0 )
        // включение объекта
    {
        // проверка готовности для всех вышестоящих объектов до корня
        cl_base * p = this -> p_head_object;
        // начать с головного объекта, текущий объект не проверяется

        while ( p != nullptr )
            // пока не достигнут корень
        {
            if ( p -> status == 0 )
                // обнаружен вышестоящий объект в выключенном состоянии
            {
                return;
            }
            // установка готовности для текущего объекта невозможна
        }

        p = p -> get_head ( );
        // переход к головному объекту (подъём на уровне выше)
    }

    this -> status = state;
    // установка состояния готовности текущего объекта
}
else
    // выключение объекта (передано нулевое значение)
{
    this -> status = 0;
    // сброс состояния готовности текущего объекта

    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )           //

```

```

цикл по всем подчинённым объектам
{
    subordinate_objects [ i ] -> set_ready ( 0 );           //
    рекурсивный вызов метода set_ready для отключения подчинённых объектов
}
}

bool cl_base :: set_head ( cl_base * new_head )
    // метод переопределения головного объекта для текущего объекта
{
    if ( this -> p_head_object == nullptr || new_head == nullptr )           //
        переопределение головного объекта для корневого объекта недопустимо и нельзя
        создать второй корневой объект
        {
            return false;
        }

    cl_base * ancestor = new_head;
        // инициализировать указатель ancestor адресом нового головного

    while ( ancestor != nullptr )
        // пока не достигнут корень
    {
        if ( ancestor == this )
            // новый головной принадлежит ветке текущего
        {
            return false;                                           //
        }
        вернуть ложь
        ancestor = ancestor -> get_head ( );
        // переход к головному объекту
    }

    if ( new_head -> get_subobject_by_name ( this -> get_name ( ) ) !=
        nullptr )
    {
        // у
        нового головного уже есть подчинённый с таким же именем
        return false;
        // дуближ имени у головного, вернуть ложь
    }

    for ( int i = 0; i < this -> p_head_object -> get_subordinate_count ( );
        i++ )
    {
        //
        удаление адреса текущего объекта из списка адресов подчинённых объектов
        старого головного объекта
        if ( this -> p_head_object -> subordinate_objects [ i ] == this )
        {
            //
            найден текущий объект в списке старого головного
            this -> p_head_object -> subordinate_objects.erase ( this ->
            p_head_object -> subordinate_objects.begin ( ) + i );
            break;
            // выход из цикла
        }
    }
}

```

```

    }

    this -> p_head_object = new_head;
    // установить новый головной объект
    new_head -> subordinate_objects.push_back ( this );
    // добавить адрес текущего объекта в список адресов подчинённых объектов
    // нового головного объекта
    return true;
    // вернуть истину
}

void cl_base :: delete_subobject ( string name )
    // метод удаления подчинённого объекта по наименованию
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
    {
        if ( subordinate_objects [ i ] -> get_name ( ) == name )
            {
                // найден подчинённый объект с указанным именем
                delete subordinate_objects [ i ];
                // удалить объект
                subordinate_objects.erase ( subordinate_objects.begin ( ) + i );
                return;
            }
        // завершить метод
    }
    // если объект не найден, метод завершает работу
}

cl_base * cl_base :: get_subobject_by_coordinate ( string coordinate ) //
    // метод получения объекта по координате
{
    if ( coordinate.empty ( ) )
        // координата - пустая строка
        {
            return nullptr;
            // вернуть значение нулевого указателя
        }

    if ( coordinate == "." )
        // текущий объект: координата равна "."
        {
            return this;
            // вернуть адрес текущего объекта
        }

    cl_base * root = this;
    // поиск корневого объекта. инициализировать указатель root адресом
    // текущего объекта
    while ( root -> get_head ( ) != nullptr )
        // пока есть головной объект
        {
            root = root -> get_head ( );
            // подняться на уровень вверх
        }
}

```

```

    if ( coordinate == "/" )
    // корневой объект: координата равна "/"
    {
        return root;
    // вернуть адрес корневого объекта
    }

    if ( coordinate.length ( ) >= 2 && coordinate[0] == '/' && coordinate[1]
== '/' )
    {
        //
        поиск по уникальному имени от корня //имя
        return root -> get_subobject_on_tree ( coordinate.substr ( 2 ) );//
        вернуть результат поиска по дереву
    }

    if ( coordinate[0] == '.' )
        // поиск по уникальному имени от текущего .имя
    {
        return this -> get_subobject_on_branch ( coordinate.substr ( 1 ) );//
        вернуть результат поиска на ветке от текущего
    }

    //
    обработка абсолютного /путь/к/объекту или относительного путь/к/объекту
    stringstream ss ( coordinate );
    // создание потока для разбора строки
    string part;
    // хранение части пути
    cl_base * cur;
    // текущий объект при обходе

    if ( coordinate[0] == '/' )
        // абсолютный путь
    {
        ss.get ( );
        // пропуск первого слеша
        cur = root;
        // начинаем от корня
    }
    else
        //
    относительный путь
    {
        cur = this;
        // начинаем от текущего
    }

    while ( getline ( ss, part, '/' ) )
        // разделение строки по слешу
    {
        if ( part.empty ( ) )
            // пропуск пустых частей
        {
            break;
        }
    }

```

```

        cl_base * found = cur -> get_subobject_by_name ( part );           //
поиск подчинённого с таким именем

        if ( found == nullptr )
            // объект не найден
            {
                return nullptr;
            }
        // вернуть значение нулевого указателя

        cur = found;
        // переход к найденному объекту
    }

    return cur;
    // вернуть адрес найденного объекта
}

int cl_base :: get_class_number ( )
    // виртуальный метод для получения номера класса объекта
{
    return 0;
    // возврат 0
}

void cl_base :: set_connect ( TYPE_SIGNAL p_signal, cl_base * p_object,
TYPE_HANDLER p_ob_hendler )// метод установления связи между сигналом и
обработчиком
{
    for ( int i = 0; i < (int) connects.size ( ); i++ )
        // цикл для исключения повторного установления связи
        {
            if ( connects [ i ] -> p_signal == p_signal      &&
                // проверка совпадения указателя на метод сигнала
                connects [ i ] -> p_cl_base == p_object      &&
                // проверка совпадения указателя на целевой объект
                connects [ i ] -> p_hendler == p_ob_hendler ) //
                проверка совпадения указателя на метод обработчика
            {
                return;
                // связь уже существует
            }
        }

    o_sh * p_value = new o_sh ( );
    // создание динамического объекта структуры для хранения информации о
связи
    p_value -> p_signal = p_signal;
    // установка указателя на метод сигнала
    p_value -> p_cl_base = p_object;
    // установка указателя на целевой объект
    p_value -> p_hendler = p_ob_hendler;
    // установка указателя на метод обработчика

    connects.push_back ( p_value );

```

```

        // добавление новой связи в вектор
    }

void cl_base :: delete_connect ( TYPE_SIGNAL p_signal, cl_base * p_object,
TYPE_HANDLER p_ob_hendler )// метод удаления связи между сигналом и
обработчиком
{
    for ( int i = 0; i < (int) connects.size ( ); i++ )
        // цикл по всем связям
        {
            if ( connects [ i ] -> p_signal == p_signal      &&
                // проверка совпадения указателя на метод сигнала
                connects [ i ] -> p_cl_base == p_object      &&
                // проверка совпадения указателя на целевой объект
                connects [ i ] -> p_hendler == p_ob_hendler ) //
                // проверка совпадения указателя на метод обработчика
                {
                    delete connects [ i ];
                    // удалить структуру связи
                    connects.erase ( connects.begin ( ) + i );
                    // удалить элемент из вектора
                    return;
                }
            // завершить метод
        }
}

void cl_base :: emit_signal ( TYPE_SIGNAL p_signal, string & s_command )//
метод выдачи сигнала с передачей строковой переменной
{
    if ( this -> status == 0 )
    {
        return;
    }

    TYPE_HANDLER    p_handler;
    // объявление переменной для хранения указателя на метод обработчика
    cl_base          * p_object;
    // объявление переменной для хранения указателя на целевой объект

    ( this ->* p_signal ) ( s_command );
    // вызов метода сигнала с передачей строковой переменной по ссылке

    for ( int i = 0; i < (int) connects.size ( ); i++ )
        // цикл по всем связям сигнал-обработчик текущего объекта
        {
            if ( connects [ i ] -> p_signal == p_signal )
                // определение допустимого обработчика - совпадает ли метод сигнала
                {
                    p_object = connects [ i ] -> p_cl_base;
                    // получение указателя на целевой объект

                    if ( p_object -> status != 0 )
                        // проверка готовности целевого объекта
                        {
                            p_handler = connects [ i ] -> p_hendler; //

```

```

    получение указателя на метод обработчика
        ( p_object ->* p_handler ) ( s_command ); //
    вызов метода обработчика целевого объекта с передачей строки по значению
    }
}
}

string cl_base :: get_absolute_path ( )
    // метод определения абсолютного пути до текущего объекта
{
    string path = "";
    // инициализировать строковую переменную path пустой строкой
    cl_base * p = this;
    // инициализировать указатель на объект базового класса p адресом
    // текущего объекта

    if ( p -> get_head ( ) == nullptr )
    {
        return "/";
    }

    while ( p -> get_head ( ) != nullptr )
        // пока не достигнут корень
    {
        path = "/" + p -> get_name ( ) + path;
        // добавить имя в начало пути
        p = p -> get_head ( );
        // переход к головному объекту
    }

    return path;
    // вернуть абсолютный путь
}

```

5.14 Файл cl_base.h

Листинг 14 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H

#include <string>
#include <vector>

#define SIGNAL_D( signal_f ) ( TYPE_SIGNAL ) ( & signal_f )
    // макрос для получения указателя на метод сигнала объекта
#define HENDLER_D( hendler_f ) ( TYPE_HANDLER ) ( & hendler_f )
    // макрос для получения указателя на метод обработчика объекта

```

```

using namespace std;

class cl_base;
    // объявление класса cl_base для использования его указателей в
    // структуре o_sh

typedef void ( cl_base :: * TYPE_SIGNAL ) ( string & );
    // тип указателя на метод сигнала объекта
typedef void ( cl_base :: * TYPE_HANDLER ) ( string );
    // тип указателя на метод обработчика объекта

struct o_sh
    // Структура задания одной связи сигнал-обработчик
{
    TYPE_SIGNAL p_signal;
    // Указатель на метод сигнала
    cl_base * p_cl_base;
    // Указатель на целевой объект
    TYPE_HANDLER p_hendler;
    // Указатель на метод обработчика
};

class cl_base
{
private:
    string s_object_name;
    // хранение наименования объекта
    cl_base * p_head_object;
    // хранение адреса на головной объект
    vector < cl_base * > subordinate_objects;
    // динамический массив адресов подчинённых объектов
    vector < o_sh * > connects;
    // динамический массив связей сигнал-обработчик

protected:
    int status;
    // состояние готовности объекта

public:
    cl_base ( cl_base* p_head_object, string s_object_name = "Base_object" );
    // параметризованный конструктор
    ~cl_base ( );
    // деструктор для удаления текущего объекта и всех его подчинённых
    // объектов

    bool set_name ( string new_name );
    // редактирует имя объекта с проверкой отсутствия дуближа среди
    // подчинённых объектов у головного
    string get_name ( );
    // возвращает наименование объекта
    cl_base * get_head ( );
    // возвращает адрес головного объекта текущего объекта
    int get_subordinate_count ( );
    // возвращает количество подчинённых объектов текущего объекта
    cl_base * get_subobject_by_name ( string name );
    // возвращает адрес подчинённого объекта по его имени (если не

```

```

найден - возвращает nullptr)
    cl_base * get_subobject_by_index ( int index );
        // возвращает адрес подчинённого объекта по его индексу (от 1 до
количества подчинённых)

    cl_base * get_subobject_on_branch ( string name );
        // метод поиска объекта на ветке дерева иерархии от текущего по
имени
    cl_base * get_subobject_on_tree ( string name );
        // метод поиска объекта на всём дереве иерархии по имени
    void print_tree ( int depth = 0 );
        // метод вывода иерархии объектов (дерева или ветки) от текущего
объекта
    void print_ready ( int depth = 0 );
        // метод вывода иерархии объектов и состояний их готовности от
текущего объекта
    void set_ready ( int state );
        // метод установки состояния готовности для текущего объекта

    bool set_head ( cl_base * new_head );
        // метод переопределения головного объекта для текущего объекта
    void delete_subobject ( string name );
        // метод удаления подчинённого объекта по наименованию
    cl_base * get_subobject_by_coordinate ( string coordinate );
        // метод получения объекта по координате

    void set_connect ( TYPE_SIGNAL p_signal, cl_base * p_object, TYPE_HANDLER
p_ob_hendler ); // метод установления связи между сигналом текущего объекта
и обработчиком целевого объекта
    void delete_connect ( TYPE_SIGNAL p_signal, cl_base * p_object,
TYPE_HANDLER p_ob_hendler ); // метод удаления (разрыва) связи между сигналом
текущего объекта и обработчиком целевого объекта
    void emit_signal ( TYPE_SIGNAL p_signal, string & s_command );
        // метод выдачи сигнала от текущего объекта с передачей строковой
переменной, включенный объект может выдать или обработать сигнал

    string get_absolute_path ( );
        // метод определения абсолютного пути до текущего объекта

    virtual int get_class_number ( );
        // виртуальный метод для получения номера класса объекта
};

#endif

```

5.15 Файл main.cpp

Листинг 15 – main.cpp

```
#include <stdlib.h>
```

```
#include <stdio.h>

#include "cl_application.h"
using namespace std;

int main()
{
    cl_application ob_cl_application ( nullptr ); // создание корневого
    объекта
    ob_cl_application.build_tree_objects ( );      // конструирование системы,
    построение дерева объектов
    return ob_cl_application.exec_app ( );        // запуск системы
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 31.

Таблица 31 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
<pre> appls_root / object_s1 3 / object_s2 2 /object_s2 object_s4 4 / object_s13 5 /object_s2 object_s6 6 /object_s1 object_s7 2 endtree /object_s2/object_s4 /object_s2/object_s6 /object_s2 /object_s1/object_s7 / /object_s2/object_s4 /object_s2/object_s4 / end_of_connections EMIT /object_s2/object_s4 Send message 1 EMIT /object_s2/object_s4 Send message 2 EMIT /object_s2/object_s4 Send message 3 EMIT /object_s1 Send message 4 END </pre>	<pre> Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 </pre>	<pre> Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 </pre>
<pre> appls_root / object_a 2 / object_b 3 endtree end_of_connections SET_CONNECT /object_a /object_b </pre>	<pre> Object tree appls_root object_a object_b Signal from /object_a Signal to /object_b </pre>	<pre> Object tree appls_root object_a object_b Signal from /object_a Signal to /object_b </pre>

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
EMIT /object_a Hello DELETE_CONNECT /object_a /object_b EMIT /object_a Hello again SET_CONDITION /object_b 0 SET_CONNECT /object_a /object_b EMIT /object_a After disable END	Text: Hello (class: 2) Signal from /object_a Signal from /object_a	Text: Hello (class: 2) Signal from /object_a Signal from /object_a
appls_root / object_s1 3 / object_s2 2 /object_s2 object_s4 4 / object_s13 5 /object_s2 object_s6 6 /object_s1 object_s7 2 endtree /object_s2/object_s4 /object_s2/object_s6 /object_s2 /object_s1/object_s7 / /object_s2/object_s4 /object_s2/object_s4 / end_of_connections EMIT /object_s2/object_s4 Send message 1 EMIT /object_s2/object_s4 Send message 2 EMIT /object_s2/object_s4 Send message 3 EMIT /object_s1 Send message 4 DELETE_CONNECT /object_s2/object_s4 / EMIT /object_s2/object_s4 Send message 2 SET_CONDITION /object_s2/object_s4 0	Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 Signal from /object_s2/object_s4 Signal to	Object tree appls_root object_s1 object_s7 object_s2 object_s4 object_s6 object_s13 Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 1 (class: 4) Signal to / Text: Send message 1 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 2 (class: 4) Signal to / Text: Send message 2 (class: 4) Signal from /object_s2/object_s4 Signal to /object_s2/object_s6 Text: Send message 3 (class: 4) Signal to / Text: Send message 3 (class: 4) Signal from /object_s1 Signal from /object_s2/object_s4 Signal to

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
<pre> EMIT /object_s2/object_s4 Send message 3 SET_CONNECT /object_s1 /object_s2/object_s6 EMIT /object_s1 Send message 4 SET_CONDITION /object_s1/object_s7 0 EMIT /object_s2 Send message 48 END </pre>	<pre> /object_s2/object_s6 Text: Send message 2 (class: 4) Signal from /object_s1 Signal to /object_s2/object_s6 Text: Send message 4 (class: 3) Signal from /object_s2 </pre>	<pre> /object_s2/object_s6 Text: Send message 2 (class: 4) Signal from /object_s1 Signal to /object_s2/object_s6 Text: Send message 4 (class: 3) Signal from /object_s2 </pre>
<pre> appls_root / obj1 5 / obj2 6 endtree / /obj1 /obj1 /obj2 /obj2 / end_of_connections EMIT / Send message from root EMIT /obj1 Send message from 5 EMIT /obj2 Send message from 6 END </pre>	<pre> Object tree appls_root obj1 obj2 Signal from / Signal to /obj1 Text: Send message from root (class: 1) Signal from /obj1 Signal to /obj2 Text: Send message from 5 (class: 5) Signal from /obj2 Signal to / Text: Send message from 6 (class: 6) </pre>	<pre> Object tree appls_root obj1 obj2 Signal from / Signal to /obj1 Text: Send message from root (class: 1) Signal from /obj1 Signal to /obj2 Text: Send message from 5 (class: 5) Signal from /obj2 Signal to / Text: Send message from 6 (class: 6) </pre>
<pre> appls_root / obj 2 / obj 3 /nonexistent obj 2 endtree end_of_connections EMIT /nonexistent Send message SET_CONNECT / /nonexistent SET_CONDITION /nonexistent 0 END </pre>	<pre> / Dubbing the names of subordinate objects Object tree appls_root obj The head object /nonexistent is not found </pre>	<pre> / Dubbing the names of subordinate objects Object tree appls_root obj The head object /nonexistent is not found </pre>
<pre> appls_root endtree end_of_connections EMIT /object_a Hello SET_CONNECT /object_a /object_b SET_CONNECT / </pre>	<pre> Object tree appls_root Object /object_a not found Object /object_a not found Handler object </pre>	<pre> Object tree appls_root Object /object_a not found Object /object_a not found Handler object </pre>

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
/object_b SET_CONDITION /object_a 1 SET_CONDITION / 0 EMIT / Hello root SET_CONDITION / 1 EMIT / Hello again END	/object_b not found Object /object_a not found Signal from /	/object_b not found Object /object_a not found Signal from /

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была соблюдена четкая последовательность этапов разработки. Работа началась с описания метода решения задачи, после чего были составлены алгоритмы для каждого метода и функции. На основе этих алгоритмов была создана блок-схема, которая послужила основой для написания программного кода.

В процессе программирования на практике было изучено, как конструировать иерархическое дерево объектов. В основе этой структуры лежит разработка базового класса. Он хранит имя объекта, адрес головного объекта, состояние его готовности, а также списки адресов подчиненных объектов и связей между методами-сигналами и методами-обработчиками. От этого базового класса были созданы 6 производных классов: один стал корневым для всего дерева, а остальные использовались для построения других объектов.

Далее было освоено, как организовывать поиск объектов по абсолютным и относительным координатам, как управлять состоянием готовности объектов с автоматическим отключением всех подчинённых при отключении головного. Затем был освоен механизм установки и разрыва связей между объектами, позволяющий передавать строковые сообщения от одного объекта к другому через сигналы и обработчики. Также было изучено, как обрабатывать пользовательские команды, проверять корректность входных данных и выводить диагностические сообщения об ошибках. Написание кода было завершено созданием работающей программы, полностью моделирующей иерархическую систему объектов с механизмом сигналов и обработчиков. Поставленные в курсовой работе задачи выполнены в полном объёме, а её цель успешно достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).